SOC LEVEL 1 · BLUE TEAM PROJECT

Wazuh SOC L1

Detection & Investigation Lab

Telemetry, detection engineering and evidence-based investigation on a Wazuh, Sysmon and Windows 10 lab, documented in eleven phases, including file integrity monitoring and vulnerability detection.

PHASES 1–11 · MASTER DOCUMENTATION

AUTHOR	Waqar
PROGRAMME	BS Computer Science (Cybersecurity), University of Buner

PLATFORM	Wazuh 4.14.7 · Windows 10 Pro endpoint with Sysmon
SCOPE	Fundamentals, Windows and Sysmon telemetry, custom detection rules, process and PowerShell hunting, authentication hunting, network monitoring, MITRE ATT&CK mapping, rule review, alert validation, file integrity monitoring, vulnerability detection and remediation
EDITION	October 2026 · Final report

> *Reading guide: every claim is tied to evidence. Hypothetical scenarios, unresolved problems and untested ideas are labelled as such. User, host and folder names are redacted.*

Contents

1. Project Overview..... 3
2. Lab Environment..... 7
3. **Phase 1: Wazuh Fundamentals and SOC Investigation Foundation**..... 10
4. **Phase 2: Windows, Sysmon and PowerShell Investigation**..... 12
5. **Phase 3: Detection Engineering and Threat Hunting**..... 14
6. **Phase 4: Advanced Process Investigation, PowerShell Hunting and Event Correlation**..... 19
7. **Phase 5: Windows Authentication Hunting and Authentication Correlation**..... 25
8. **Phase 6: Network Security Monitoring and Custom Rule Troubleshooting**..... 30
9. **Phase 7: MITRE ATT&CK Mapping and Detection Validation**..... 39
10. **Phase 8: Advanced Detection Engineering**..... 43
11. **Phase 9: Detection Validation and SOC Investigation**..... 52
12. **Phase 10: File Integrity Monitoring (FIM) Investigation**..... 58
13. **Phase 11: Vulnerability Detection and Remediation**..... 68
14. Custom Rules Reference..... 77
15. Commands Reference..... 79
16. Key Lessons..... 81
17. MITRE ATT&CK Coverage..... 82
18. Detection Catalogue..... 83
19. Sample Incident Report (Lab Exercise)..... 85
20. Evidence Index..... 86

1. Project Overview

This project documents a hands-on SOC Level 1 lab built around Wazuh. The goal was not to click through a dashboard, but to practise the real analyst workflow: collect endpoint telemetry, understand how Wazuh turns events into alerts, investigate those alerts using evidence, and write custom detections.

The core mindset of the project: **an alert is a starting point for investigation, not proof of an attack.**

The workflow practised throughout the lab:

```
Windows Endpoint -> Sysmon -> Wazuh Agent -> Wazuh Manager -> Decoder -> Rules -> Alert
-> SOC Investigation -> Evidence -> Verdict -> Response / Escalation
```

Phase	Focus	Outcome
Phase 1	Wazuh fundamentals and SOC investigation foundation	Understood architecture, event vs alert, and the investigation method; completed two investigations
Phase 2	Windows, Sysmon and PowerShell investigation	Learned key event IDs and correlated process, file, DNS and network telemetry
Phase 3	Detection engineering and threat hunting	Wrote and debugged two custom rules; hunted persistence; completed blind investigations
Phase 4	Advanced process investigation, PowerShell hunting and event correlation	Reconstructed a process tree, hunted encoded PowerShell and correlated Events 1, 11 and 22 by ProcessGuid, documenting that Event 3 was not observed
Phase 5	Windows authentication hunting and authentication correlation	Investigated failed (4625) and successful (4624) logons in Wazuh and correlated them as one timeline instead of isolated alerts
Phase 6	Network security monitoring: Sysmon Event 3 and custom rule troubleshooting	Read Event 3 network telemetry, correlated process and DNS evidence, and documented an unresolved custom rule (100103)
Phase 7	MITRE ATT&CK mapping and detection validation	Verified Sysmon Event 3 collection and decoding, mapped events to ATT&CK only where evidence allowed, and documented why searches for rules 61605 and 100103 found no alert

Phase	Focus	Outcome
Phase 8	Advanced detection engineering	Reviewed the event-to-alert pipeline and rule hierarchy, analysed rules 100101 and 100102 (benign triggers, blind spots), compared the live rule file with the documentation, and recorded rule 100103 and the archive setting as unresolved
Phase 9	Detection validation and SOC investigation	Investigated a real rule 100102 alert end to end, separated observed facts from inferences, and documented the missing execution evidence as an explicit gap instead of assuming an outcome
Phase 10	File integrity monitoring (FIM) investigation	Verified Wazuh FIM on a controlled file: creation, modification and deletion were all detected, and the recorded sizes, timestamps and hashes were cross-checked; one older FIM alert was recorded as inconclusive
Phase 11	Vulnerability detection and remediation	Investigated a High-severity 7-Zip finding (CVE-2026-48111), verified the installed version, updated the software, investigated a stale Wazuh inventory and confirmed the finding no longer appeared; the cause of the stale record is recorded as unproven

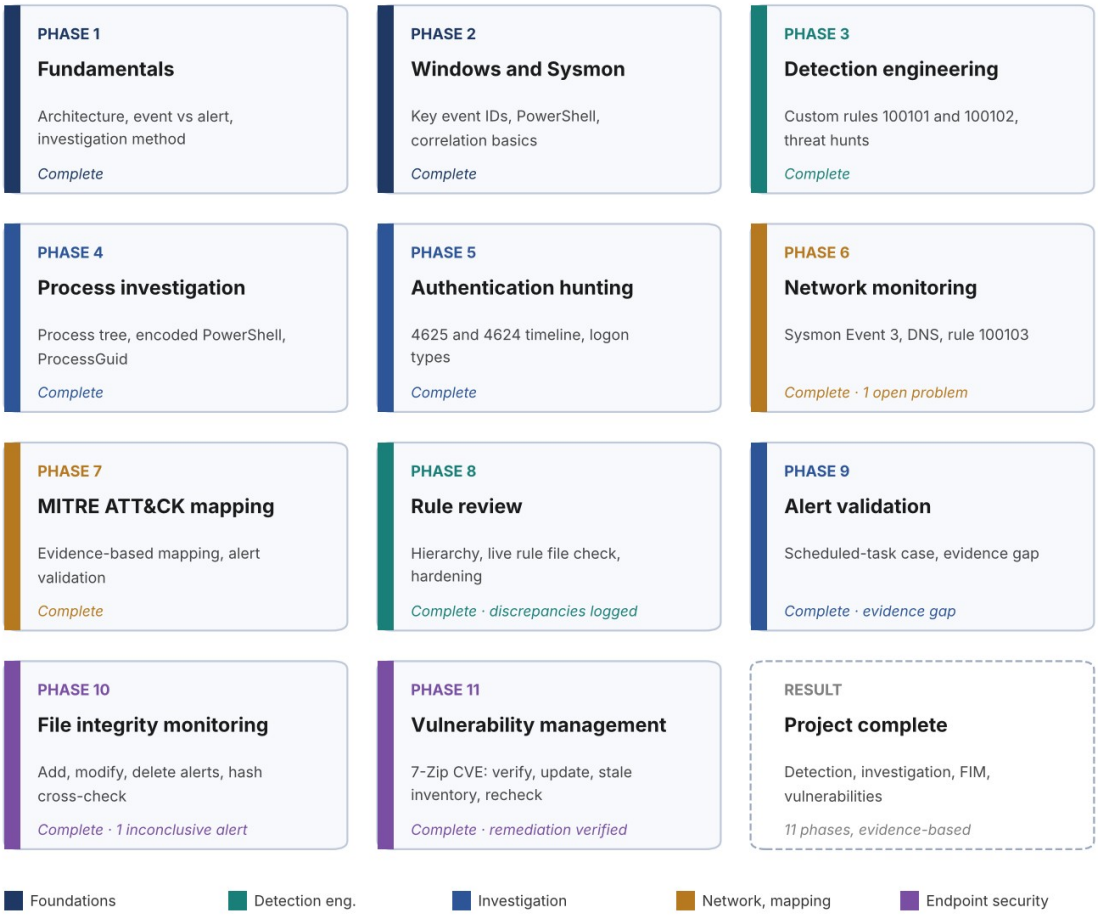


Figure 1.1: The eleven phases at a glance, grouped by skill area.

11	3	66	11
Phases completed	Custom rules written (2 confirmed, 1 open)	Figures and screenshots	ATT&CK techniques mapped

1.1 Executive summary

What was built. A working SOC L1 detection and investigation lab: a Windows 10 endpoint with Sysmon and the Wazuh agent, reporting to a Wazuh 4.14.7 manager. Eleven phases cover fundamentals, Windows and Sysmon telemetry, detection engineering, process and PowerShell investigation, authentication hunting, network monitoring, MITRE ATT&CK mapping with detection validation, advanced detection engineering, an evidence-based alert investigation, file integrity monitoring, and vulnerability detection with remediation.

What was achieved.

- Three custom detection rules written against real telemetry (100101, 100102, 100103); two verified working, one documented as unresolved.

- Real troubleshooting recorded: a regex and a field-name bug found and fixed in rule 100102, validated with `wazuh-analysisd -t`.
- Events correlated by ProcessGuid across Sysmon Events 1, 11, 22 and 3, and authentication events 4625 and 4624 read as one timeline.
- File integrity monitoring validated end to end: a file was created, modified and deleted, and every alert was cross-checked against independent evidence (sizes, hashes, timestamps).
- A full vulnerability-management cycle: a High finding was confirmed, fixed and verified closed in Wazuh, and a stale inventory record was investigated rather than ignored.
- Verdicts written with evidence, using one framework: benign, suspicious, needs further investigation, confirmed malicious.
- Honest reporting: missing telemetry, hypothetical scenarios and unresolved problems are labelled as such, never presented as results.

What is not claimed. No real attack or malware was analysed. All activity was generated by the author in an isolated lab. The detections have not been tested against adversary-simulation tooling yet.

1.2 Skills demonstrated

Skill	Where shown
Alert triage and evidence-based verdicts	Phases 1, 3, 4, 5
Sysmon and Windows event analysis (1, 3, 11, 22, 4624, 4625)	Phases 2, 4, 5, 6
Detection engineering (Wazuh rules, PCRE2, rule hierarchy, MITRE tags, rule review)	Phases 3 and 8, Section 14
Troubleshooting a detection that does not fire	Phase 3 (fixed), Phase 6 (open)
Event correlation (ProcessGuid, timeline)	Phases 4, 5, 6
Network telemetry analysis (Sysmon Event 3 and DNS)	Phase 6
File integrity monitoring and hash verification	Phase 10
Vulnerability triage, remediation and verification	Phase 11
Agent health and inventory troubleshooting	Phase 11
Technical writing and honest reporting	Whole document

2. Lab Environment

Component	Role
Windows 10 endpoint	Monitored host running Sysmon and the Wazuh agent
Sysmon	Detailed process, network, file and DNS telemetry
Wazuh agent	Collects and forwards events from the endpoint
Wazuh manager (Linux, `wazuh-server`)	Decodes events, matches rules, generates alerts, hosts the dashboard
Wazuh agent name	`WIN10-SOC-LAB` (Agent ID 001)

Item	Value
Wazuh	4.14.7 (manager and agent)
Windows	Windows 10 Pro, OS build 10.0.19045.7291 (Wazuh agent panel, Figure 13.7); file versions in Sysmon events in earlier phases carry the 19041 number
Sysmon	Event schema version 5 (from archived records); the program version was not recorded
Virtualisation	The Wazuh manager runs in a VirtualBox virtual machine
Agent to manager link	TCP 1514 to the manager (IP address redacted; Section 2.2)
Vulnerability detection	Wazuh Vulnerability Detection module with Syscollector inventory (hourly, also on agent start); 91 findings at the time of Phase 11
File integrity monitoring	Enabled on the agent (syscheck): 12-hour scans, Startup folder monitored in real time (Phase 10)

2.1 Conventions used in this document

Topic	Convention
Time zone	Sysmon `UtcTime` and archive timestamps are UTC. The Wazuh dashboard and Event Viewer show local time, UTC+5. Tables state which one they use
Redaction	User, host and folder names are blacked out in screenshots and replaced by placeholders in text before publication
Verdicts	Benign, suspicious, needs further investigation, confirmed malicious. Nothing in this lab is confirmed malicious
Honesty labels	*Hypothetical* (training scenario), *unresolved* (open problem), *hypothesis* (unconfirmed explanation)
Figure numbers	Figure N.M belongs to section N

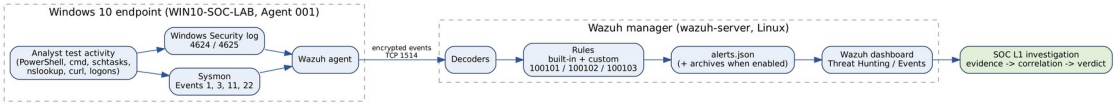


Figure 2.1: Lab architecture: endpoint telemetry (Sysmon and the Windows Security log) travels through the Wazuh agent to the manager, where decoders and rules produce alerts for the analyst.

Reading process IDs in this document. Process IDs (PIDs) are reassigned by Windows every time a process starts, so the same program shows a different PID in every test. For example, PowerShell had PID 1460 as a parent in Phase 3 and PID 1084 in Phases 4 and 6: these are different PowerShell sessions, not an inconsistency. Sysmon records PIDs in decimal. The Windows Security events in Phase 5 record the caller process in hexadecimal (`0x984` is PID 2436 in decimal). Because PIDs can be reused, correlation in this project relies on **ProcessGuid first, and uses PID, image, user and time only as supporting evidence.**

2.2 How events travel from the agent to the manager

Every detection in this report depends on one thing: the endpoint's events reaching the manager. This section describes that path. Items marked *observed* were seen in this lab; the rest describes Wazuh's documented default design and was not separately tested here.

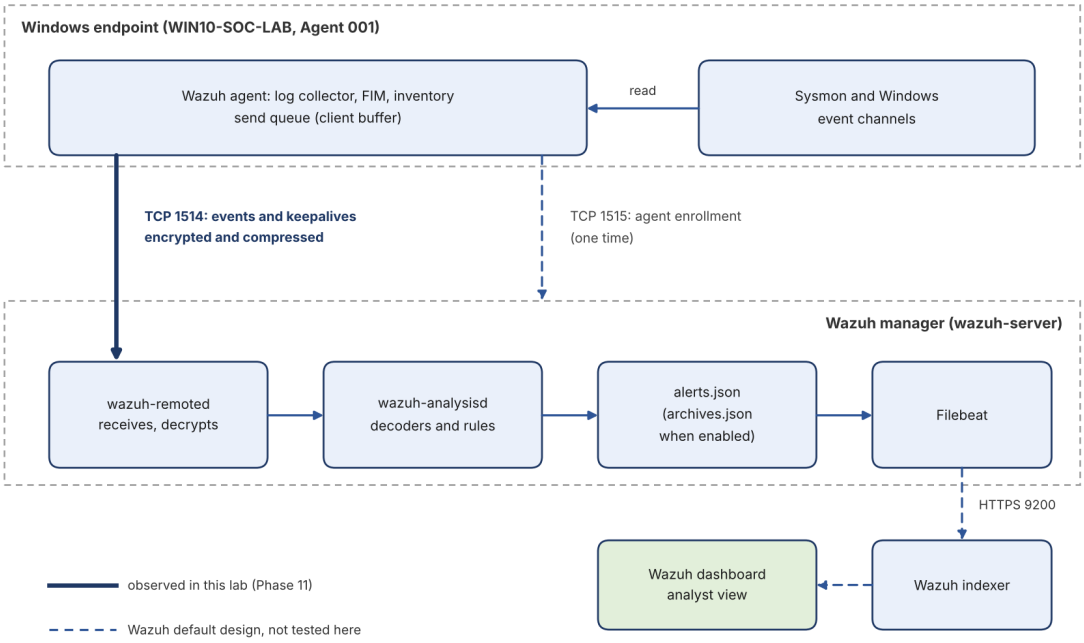


Figure 2.2: Path of an event from the Windows endpoint to the analyst. The bold arrow is the agent-to-manager channel seen in this lab (TCP 1514); dashed arrows are parts of the default Wazuh design that were not tested separately.

Channel	Protocol and port	Purpose	Status
Agent to manager (events)	TCP 1514	The agent sends collected events (Windows and Sysmon event channels, file integrity data, software inventory) and periodic keepalive messages, and receives configuration and commands	**Observed**: the agent log names `<manager-ip>:1514/tcp`, the manager listens on 1514 and `Test-NetConnection` succeeded (Section 13.7)
Agent enrollment	TCP 1515	One-time registration: the manager issues the agent ID and a key that the agent then uses to protect its messages	Default design; agent 001 is enrolled and active (Figure 3.1)
Manager to indexer	HTTPS 9200	Filebeat ships alerts to the Wazuh indexer, which the dashboard queries	Default design

The journey of one event.

1. The event is created on the endpoint: Sysmon writes to its operational log and Windows writes the Security log.
2. The agent's log collector reads those event channels (the same events the manager later decodes with the `windows_eventchannel` decoder, Phase 8). File integrity and inventory modules feed the same agent.
3. The agent places events in a send queue (the `client_buffer` setting seen in Phase 10, 5000 events) and sends them to the manager over the TCP connection to port 1514. Messages are compressed and encrypted with the key issued to that agent.
4. On the manager, `wazuh-remoted` receives and decrypts the messages and passes them to `wazuh-analysisd`.
5. `wazuh-analysisd` decodes each event and evaluates the rules. A match produces an alert in `alerts.json`; if archiving is enabled, every event is also written to `archives.json` (Sections 8.5 and 10.5).
6. Filebeat ships the alerts to the indexer and the dashboard displays them, which is where the analyst investigates.

Why this matters to an analyst. If the agent cannot reach port 1514, nothing arrives: no alerts, no file integrity events and no inventory updates. That is exactly what the connection errors in Phase 11 showed ("Lost connection with manager", "Process locked due to agent is offline"), and it is why an *active* agent status (Figure 3.1) is the first thing to check when a detection is silent. Events held in the agent's queue while it is disconnected can reach the manager late; this is one candidate, not a proven cause, for the delay between event time and archive time discussed in Section 9.2.

3. Phase 1: Wazuh Fundamentals and SOC Investigation Foundation

3.1 What Wazuh does

Wazuh is a security monitoring / XDR platform used for log collection, threat detection, File Integrity Monitoring, vulnerability detection, Security Configuration Assessment, MITRE ATT&CK mapping, endpoint monitoring and incident investigation. From a SOC view: it collects telemetry from endpoints, analyses it with decoders and rules, generates alerts, and gives the analyst a place to investigate them.

3.2 Architecture

- **Agent:** runs on the endpoint and collects Windows, Sysmon, file, security and process events.
- **Manager:** the central analysis component. Receives agent data, decodes it, matches rules, generates alerts and manages agents and configuration.
- **Decoder:** understands and structures the incoming event.
- **Rule:** decides whether the decoded event is interesting enough to alert on.
- **Alert:** produced when an event satisfies a rule.

```
Event -> Decoder -> Rule -> Alert
```

The starting point for every phase is a connected agent. The Wazuh dashboard lists the Windows endpoint as active:

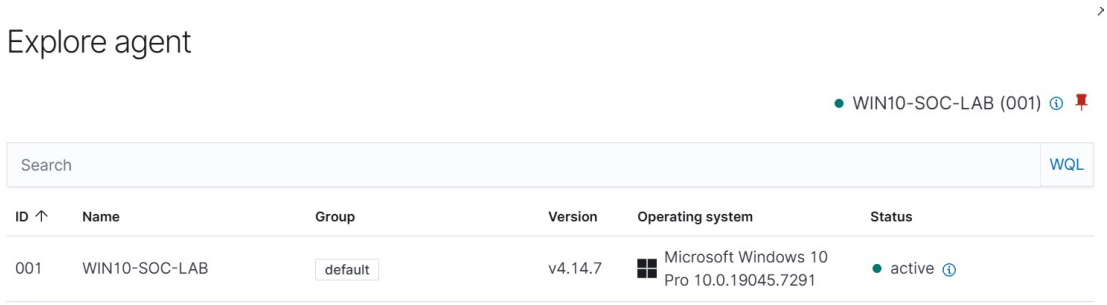


*Figure 3.1: Dashboard agent panel: `WIN10-SOC-LAB (001)`, group `default`, version v4.14.7, Microsoft Windows 10 Pro 10.0.19045.7291, status **active**. Captured on 13 October 2026, at the end of the lab.*

3.3 Key distinctions

- **Event is not Alert.** Windows Event ID 4625 is an event (a failed logon). It only becomes an alert if a Wazuh rule matches it.
- **Alert is not malware.** An alert only means “something matched this detection condition.”
- **Rule ID vs Rule Level.** The ID identifies the rule (for example 92027). The level (for example 4) is the severity assigned by the rule author. Level is not the final verdict: a level 10 alert can be legitimate and a level 4 alert can deserve investigation.

3.4 Investigation methodology

1. What actually triggered the alert?

2. What process or event caused it?
3. Who performed it?
4. What was the command line?
5. What was the parent process?
6. What happened before?
7. What happened after?
8. Are there related events?
9. Is there a legitimate explanation?
10. What is my evidence-based verdict?

Possible verdicts: Benign, False positive, Suspicious, Needs investigation, Malicious, Confirmed compromise.

3.5 Investigation 1: McAfee updater spawning cmd.exe

Alert: Windows command prompt started by an abnormal process (Rule 92052, Level 4).

Sysmon Event 1 evidence:

- Command: `cmd.exe /c DEL "C:\Program Files\McAfee\WebAdvisor\*.tmp"`
- Parent: `updater.exe` inside the McAfee WebAdvisor folder
- User: SYSTEM

Analysis: An updater launching cmd.exe looks suspicious at first glance. In context, the parent is the McAfee updater, the command only deletes temporary files, and it runs as SYSTEM, which matches normal software maintenance.

Verdict: Preliminary benign. A suspicious-looking process chain must be judged in context.

3.6 Investigation 2: failed logon (Event ID 4625)

A failed login was generated deliberately in the lab.

Field	Value
Event ID	4625
Logon type	2 (interactive)
Source	127.0.0.1
Status / Substatus	0xC000006D / 0xC000006A
Rule	60122, Level 5

Verdict: Benign, intentional lab activity. One failed login is not brute force; it could be a typo or a test. A very different picture would be 20+ failures from one source in a short time across multiple accounts, followed by a successful login (see Phase 3 validation exercises).

4. Phase 2: Windows, Sysmon and PowerShell Investigation

4.1 Windows Security events used

Event ID	Meaning
4624	Successful logon
4625	Failed logon
4634	Logoff
4648	Explicit credential use
4672	Special privileges assigned
4688 / 4689	Process creation / termination
4698 / 4699 / 4702	Scheduled task created / deleted / updated
4720 / 4726	User created / deleted
4732	Member added to local group
7045	Service installed

4.2 Sysmon events used

Event ID	Meaning
1	Process creation
3	Network connection
5	Process terminated
7	Image loaded
8	CreateRemoteThread
10	Process access
11	File create
12 / 13	Registry object / value events
22	DNS query
23	File delete
24	Clipboard change
25	Process tampering

The four that mattered most in this lab: **1 (process), 3 (network), 11 (file), 22 (DNS).**

4.3 PowerShell in context

PowerShell is legitimate, powerful and widely used by administrators, but also frequently abused by attackers. PowerShell itself is not malicious, so the context decides. Evidence to collect: parent and child process, command line, user, integrity level, PID, ProcessGuid, file creation, DNS, network connections, script content and encoded commands.

4.4 Investigation: PowerShell temporary script file (Sysmon Event 11)

PowerShell created a file named like `__PSScriptPolicyTest_....ps1` in the user's `AppData\Local\Temp` folder. Wazuh raised a high-level alert (Rule 8288, Level 15) describing an executable file dropped in a location commonly used by malware.

The conclusion was **not** "temp file means malware." The questions asked instead: what created it, why, was it executed, what did it contain, and what happened afterwards? **File creation does not prove execution.**

4.5 Investigation: encoded PowerShell commands

`-EncodedCommand` is a suspicious indicator, not proof of malicious activity. The approach: decode the command, understand the real behaviour, then investigate the surrounding evidence.

4.6 DNS and network telemetry

- **Event 22** records a DNS query. `example.com` resolved to two addresses.
- **Event 3** records a network connection attributed to a process (here PowerShell connecting to a destination on port 443).

Event 3 is process-centric telemetry, not a packet capture.

4.7 Process correlation

Events are correlated using PID, ProcessGuid, timestamp, Image, ParentImage and CommandLine:

```
PowerShell Event 1 -> same ProcessGuid -> DNS Event 22 -> same process -> network Event 3
```

Events that merely happen at about the same time must not be assumed to be related.

5. Phase 3: Detection Engineering and Threat Hunting

5.1 How custom rules build on built-in rules

```
Sysmon Event -> Decoder -> Built-in Rule -> Custom Rule -> Alert
```

The `if_sid` option makes a custom rule evaluate only events that already matched a parent rule. The built-in parent used here was Rule 61603 (Sysmon Event 1, process creation):

```
<rule id="61603" level="0">
  <if_sid>61600</if_sid>
  <field name="win.system.eventID">^1$</field>
  <description>Sysmon - Event 1: Process creation
$(win.eventdata.description)</description>
  <options>no_full_log</options>
  <group>sysmon_event1,</group>
</rule>
```

5.2 Custom Rule 100101: PowerShell spawned by cmd.exe

The rule only evaluates Sysmon Event 1 events (via 61603), requires the parent image to end with cmd.exe and the child image to end with powershell.exe, and raises a level 10 alert. The full XML is in Section 14 (Rule 100103 from Phase 6 is listed there too). **Note from Phase 8:** the live file uses looser patterns (`(?i)cmd.exe`, `(?i)powershell.exe`) than the description above; see Section 10.8.

```
<rule id="100101" level="10">
  <if_sid>61603</if_sid>
  <field name="win.eventdata.parentImage" type="pcre2">(?i)cmd.exe</field>
  <field name="win.eventdata.image" type="pcre2">(?i)powershell.exe</field>
  <description>PowerShell spawned by cmd.exe</description>
  <group>sysmon,powershell,process_creation,</group>
</rule>
```

Figure 5.1: Custom rule 100101 as saved in the lab.

5.3 Hunt: PowerShell parent-child relationship

Commands run:

```
Get-Date
Write-Host "DAY03-HUNT-TEST"
powershell.exe -NoProfile -Command "Get-Date"
```

The third command produced **Rule 92027, Level 4: PowerShell process spawned PowerShell instance.** Child PID 8896, parent PID 1460, parent PowerShell, run by the interactive lab user.

Verdict: Benign. The alert fired because of the PowerShell-to-PowerShell parent-child relationship, not because `-NoProfile` is malicious.

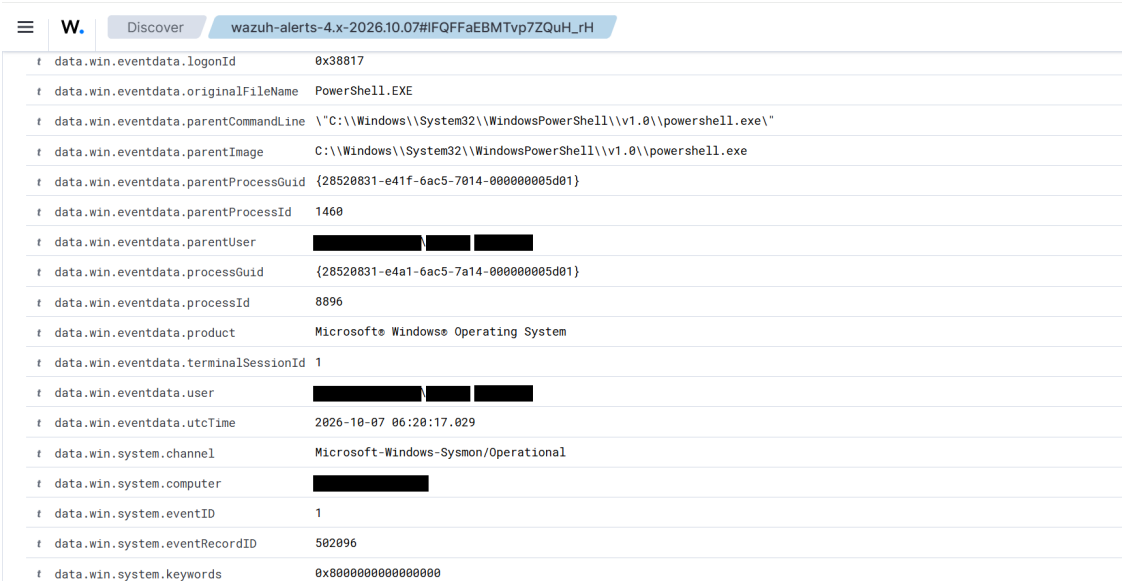


Figure 5.2: Sysmon Event 1 details in Wazuh Discover for the PowerShell-in-PowerShell alert: child PID 8896, parent PID 1460, parent image PowerShell.

5.4 Persistence hunt and Rule 100102 (with troubleshooting)

A scheduled task was created in a controlled way to simulate a persistence technique:

```
schtasks /Create /TN "Wazuh-Day03-Test" /TR "cmd.exe /c echo DAY03-PERSISTENCE-TEST" /SC ONLOGON /F
```

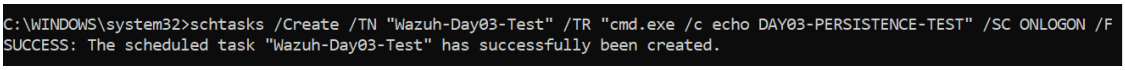


Figure 5.3: The scheduled task was created successfully.

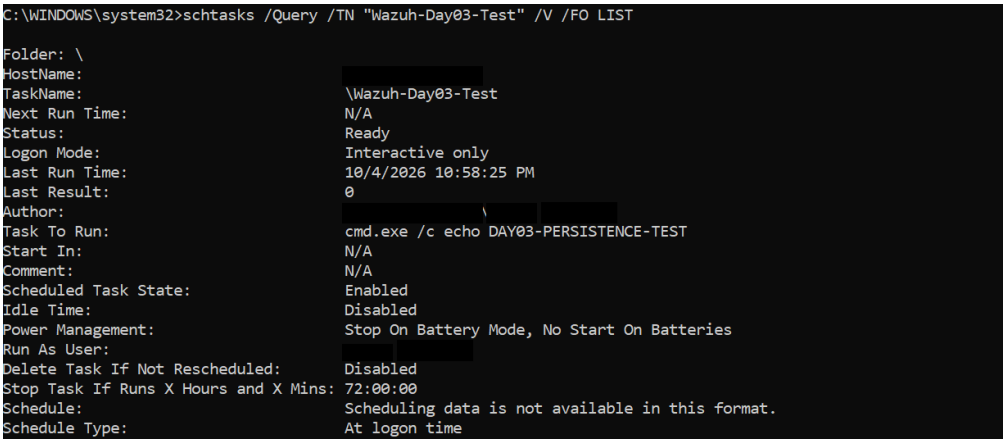


Figure 5.4: Task details from schtasks /Query: trigger At logon time, action cmd.exe /c echo DAY03-PERSISTENCE-TEST, status Ready.

Sysmon recorded it as Event 1: image schtasks.exe, parent cmd.exe, High integrity, PID 2832 (the first test, Figure 5.5). Later tests ran the same command from other shells, so the parent differs between figures: Figure 5.7 belongs to a fresh test (task `Wazuh-Day03-Test-4`) started from PowerShell. **Wazuh did not generate the custom alert.** The investigation that followed:

1. **Problem:** the first version of Rule 100102 never fired.
2. **Root cause (regex):** the pattern expected `/create` to be followed immediately by `s` or the end of the string, but the real command was `/Create /TN ...` with a space after it.
3. **Root cause (field name):** the rule used `commandline`, while the decoder writes the field as `commandLine`. (Phase 8 later found the lowercase spelling in the live file; whether the spelling matters is an open question, see Section 10.8.)
4. **Fix:** corrected the pattern to `(?i)\s/create(\s|$)` and the field to `win.eventdata.commandLine`.
5. **Validation:** checked the configuration with `sudo /var/ossec/bin/wazuh-analysisd -t`, restarted Wazuh and generated a fresh event.
6. **Result: Rule 100102, Level 10: Scheduled task created using schtasks.exe** appeared, mapped to MITRE ATT&CK T1053.005 (Scheduled Task/Job: Scheduled Task).

The lesson: **do not assume the event format. Read the real telemetry and write the detection around it.** Also, a MITRE mapping describes a technique; it does not prove malicious activity.

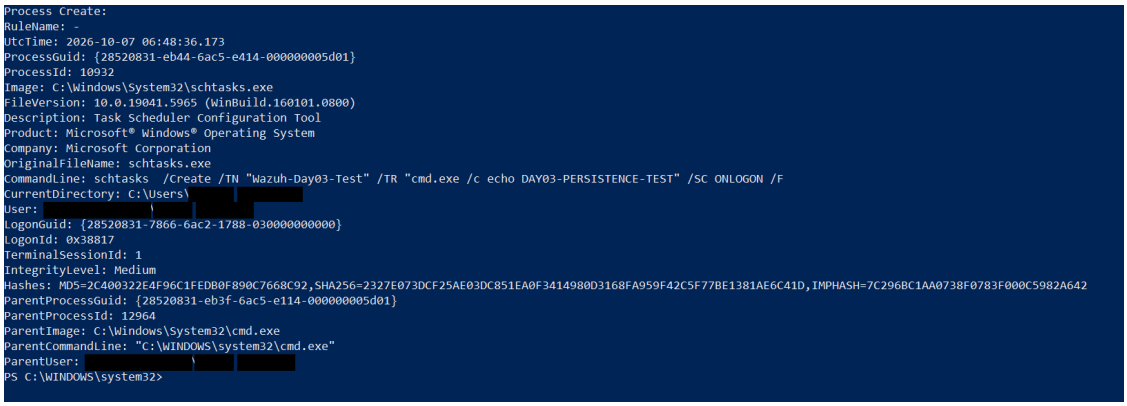


Figure 5.5: Sysmon Event 1 for the schtasks command (2026-10-07, 06:48 UTC): image schtasks.exe, parent cmd.exe.

```
<rule id="100102" level="10">
  <if_sid>61603</if_sid>
  <field name="win.eventdata.image" type="pcre2">(?i)\\schtasks\.exe$</field>
  <field name="win.eventdata.commandline" type="pcre2">(?i)\s/create(\s|$)</field>
  <description>Scheduled task created using schtasks.exe</description>
  <mitre>
      <id>T1053.005</id>
  </mitre>
  <group>sysmon,persistence,scheduled_task.</group>
</rule>
```

Figure 5.6: Rule 100102 as edited at this stage; the field name is still lowercase commandline here, which is the problem described in step 3.

t data.win.eventdata.commandLine	\"C:\\WINDOWS\\system32\\schtasks.exe\" /Create /TN Wazuh-Day03-Test-4 /TR \"cmd.exe /c echo DAY03-PERSISTENCE-TEST-4\" /SC ONLOGON /F
t data.win.eventdata.company	Microsoft Corporation
t data.win.eventdata.currentDirectory	C:\\WINDOWS\\system32\\
t data.win.eventdata.description	Task Scheduler Configuration Tool
t data.win.eventdata.fileVersion	10.0.19041.5965 (WinBuild.160101.0800)
t data.win.eventdata.hashes	MD5=2C400322E4F96C1FEDB0F890C7668C92,SHA256=2327E073DCF25AE03DC851EA0F3414980D3168FA959F42C5F77BE1381AE6C41D,IMPHASH=7C296BC1AA0738F0783F000C5982A642
t data.win.eventdata.image	C:\\Windows\\System32\\schtasks.exe
t data.win.eventdata.integrityLevel	High
t data.win.eventdata.logonGuid	{28520831-7866-6ac2-b287-030000000000}
t data.win.eventdata.logonId	0x387b2
t data.win.eventdata.originalFileName	schtasks.exe
t data.win.eventdata.parentCommandLine	\"C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe\"
t data.win.eventdata.parentImage	C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe
t data.win.eventdata.parentProcessGuid	{28520831-2875-6ac6-b817-000000005d01}
t data.win.eventdata.parentProcessId	388
t data.win.eventdata.parentUser	[redacted]
t data.win.eventdata.processGuid	{28520831-329e-6ac6-b218-000000005d01}

Figure 5.7: Alert details for the fresh test (task Wazuh-Day03-Test-4): schtasks.exe /Create, High integrity, parent PowerShell. The same task is investigated again in Phase 9 (Section 11).

5.5 Correlation hunt: PowerShell, DNS and HTTPS

```
powershell.exe -NoProfile -Command "Resolve-DnsName example.com; curl.exe
https://example.com"
```

Observed: **Event 1** (PowerShell process creation, PID 5292) and **Event 22** (DNS query for example.com). **Event 11 and Event 3 were not observed** in this test, and no network event was assumed just because curl was used.

t data.win.eventdata.commandLine	\"C:\\WINDOWS\\System32\\WindowsPowerShell\\v1.0\\powershell.exe\" -NoProfile -Command \"Resolve-DnsName example.com; curl.exe https://example.com\"
t data.win.eventdata.company	Microsoft Corporation
t data.win.eventdata.currentDirectory	C:\\Users\[redacted]
t data.win.eventdata.description	Windows PowerShell
t data.win.eventdata.fileVersion	10.0.19041.3996 (WinBuild.160101.0800)
t data.win.eventdata.hashes	MD5=2E5A8590CF6848968FC23DE3FA1E25F1,SHA256=9785001B0DCF755EDDB8AF294A373C0B87B2498660F724E76C4D53F9C217C7A3,IMPHASH=3D08F4848535206D772DE145804FF4B6
t data.win.eventdata.image	C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe
t data.win.eventdata.integrityLevel	Medium
t data.win.eventdata.logonGuid	{28520831-7866-6ac2-1788-030000000000}
t data.win.eventdata.logonId	0x38817
t data.win.eventdata.originalFileName	PowerShell.EXE
t data.win.eventdata.parentCommandLine	\"C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe\"
t data.win.eventdata.parentImage	C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe
t data.win.eventdata.parentProcessGuid	{28520831-e41f-6ac5-7014-000000005d01}
t data.win.eventdata.parentProcessId	1460
t data.win.eventdata.parentUser	[redacted]
t data.win.eventdata.processGuid	{28520831-3883-6ac6-3a19-000000005d01}

Figure 5.8: Event 1 for the correlation test: PowerShell running Resolve-DnsName example.com and curl.exe, ProcessGuid {28520831-3883-6ac6-3a19-000000005d01}.

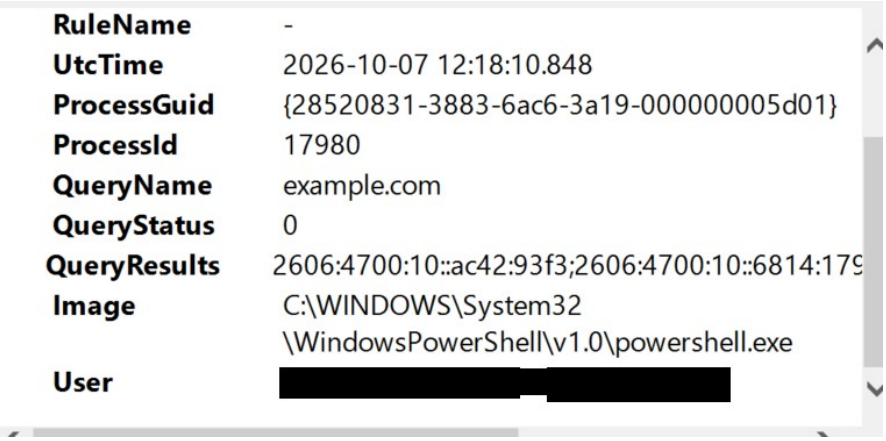


Figure 5.9: Event 22 for the same ProcessGuid: DNS query for example.com (PID 17980).

5.6 Design decision: no high-severity correlation rule

It was considered whether Event 1 + 11 + 22 + 3 together should become one level 10–12 alert, and the decision was **no**. More events does not automatically mean more malicious: PowerShell, then a DNS lookup, then an HTTPS request can be completely normal. A good correlation rule needs high-signal suspicious characteristics, not just several events in a row.

5.7 Validation exercises: blind investigations

Scenarios were investigated without being told the answer.

Scenario	Evidence	Verdict and reasoning
1	Rule 92027, PowerShell spawning PowerShell, `-NoProfile -Command Get-Date`	**Benign.** Harmless command, parent-child alert only
2	schtasks /Create, ONLOGON, PowerShell, EncodedCommand, High integrity	**Suspicious, investigate immediately.** Decode the command, inspect what the task executes, check activity before and after, escalate if supported
3	27 failed logins, multiple accounts, same source, about 2 minutes, then a successful administrator login	**Brute force / password spraying hypothesis, not confirmed compromise.** Source 10.10.20.55 is a private/internal address, so check whether remote login is expected, investigate the successful login, then look for persistence and suspicious processes
4	`-EncodedCommand`, `-ExecutionPolicy Bypass`, High integrity, cmd.exe parent; decoded to `Get-Date`	**Likely benign, technique needs validation.** A suspicious method does not mean a malicious payload

6. Phase 4: Advanced Process Investigation, PowerShell Hunting and Event Correlation

Phase 4 moves from reading alerts to running full SOC L1 investigations: reconstructing a process tree, hunting encoded PowerShell, and correlating process, file and DNS telemetry by ProcessGuid. The activity was run in the lab on 8 October 2026 (UTC) and followed one workflow: **Alert, Evidence, Correlation, Assessment, Verdict.** Usernames, hostnames and profile paths are replaced with placeholders for publication.

6.1 Process investigation: cmd.exe launching PowerShell

Command run:

```
cmd.exe /c powershell.exe -NoProfile -Command "Get-Date"
```

Sysmon Event 1 recorded the process creation:

Field	Value
Time (UTC)	2026-10-08 05:12:30
Process ID / Image	14396 / cmd.exe
Command line	`cmd.exe /c powershell.exe -NoProfile -Command Get-Date`
User / Integrity	`lab-user` / Medium
Parent	PowerShell, PID 1084

The command line shows cmd.exe being used to launch PowerShell, but the PowerShell command itself is only `Get-Date`, so no malicious payload was identified.

Verdict: Benign controlled activity. An unusual process relationship justifies a closer look, but unusual is not the same as malicious.



Figure 6.1: Sysmon Event 1 for cmd.exe (PID 14396), parent PowerShell (PID 1084), 2026-10-08 05:12:30 UTC, Medium integrity.

6.2 Process tree reconstruction

A deeper chain was created on purpose:

```
cmd.exe /c powershell.exe -NoProfile -Command "cmd.exe /c echo DAY04-PROCESS-TREE"
```

Three Sysmon Event 1 records were observed:

Time (UTC)	PID	Image	Parent PID	Parent image
05:33:13.863	16212	cmd.exe	1084	PowerShell
05:33:14.001	1540	powershell.exe	16212	cmd.exe
05:33:15.261	6932	cmd.exe	1540	PowerShell

Reconstructed tree:

```
PowerShell (PID 1084)
  -> cmd.exe (PID 16212)
       -> PowerShell (PID 1540)
            -> cmd.exe (PID 6932): echo DAY04-PROCESS-TREE
```

Custom Rule 100101 (PowerShell spawned by cmd.exe) fired for the PowerShell process in the middle of the chain, while the other two events produced no alert. **Telemetry can exist without an alert:** Sysmon recorded all three events, but Wazuh alerts only when a rule matches.

Verdict: Benign controlled laboratory activity. No encoded payload, download, suspicious file, network connection, persistence or privilege escalation was observed. The same chain in an unknown environment would still deserve investigation.

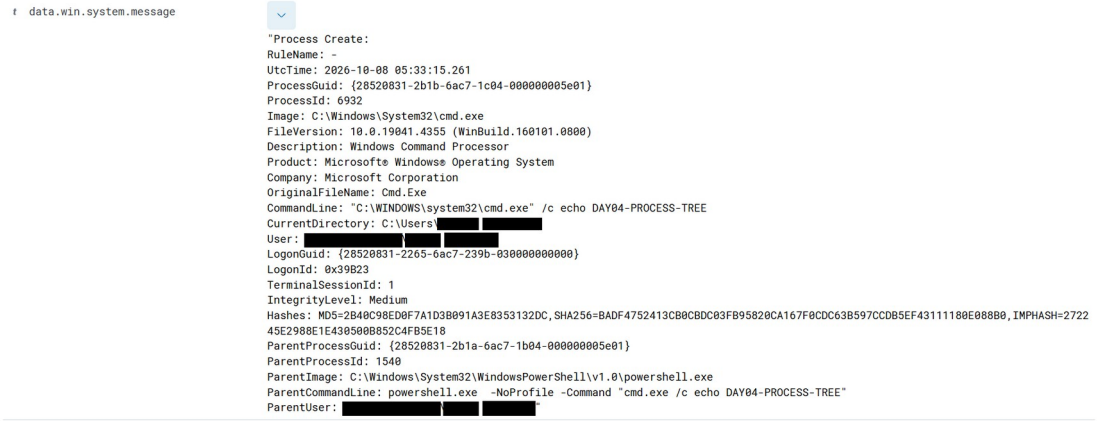


Figure 6.2: Sysmon Event 1 for the last process in the chain: cmd.exe (PID 6932), parent PowerShell (PID 1540), command echo DAY04-PROCESS-TREE.

6.3 Encoded PowerShell hunt

A Base64 payload was generated, then run with `-EncodedCommand`:

```
$cmd = "Write-Host 'DAY04-ENCODED-TEST'"
[Convert]::ToBase64String([Text.Encoding]::Unicode.GetBytes($cmd))

powershell.exe -NoProfile -EncodedCommand <base64-string>
```

Sysmon Event 1 evidence: time 05:50:07 UTC, PID 5920, ProcessGuid `{28520831-2f0f-6ac7-8a04-000000005e01}`, user `lab-user`, Medium integrity, parent PowerShell (PID 1084).

The case was worked as a full investigation: alert, process, command line, decode the payload, file activity, DNS, network, persistence, context, verdict.

- `-EncodedCommand` was treated as a suspicious characteristic, not as malware.
- The payload decoded to `Write-Host 'DAY04-ENCODED-TEST'`, which only prints text.
- No related Event 11, Event 22 or Event 3 was observed for this process, and no persistence or suspicious child process was found.

Verdict: Benign controlled laboratory activity. Encoded PowerShell is not automatically malicious: decode it, understand the behaviour, check supporting telemetry, then decide.



Figure 6.3: Encoded PowerShell Event 1: PID 5920, ProcessGuid {28520831-2f0f-6ac7-8a04-000000005e01}, command line with -NoProfile -EncodedCommand.

6.4 Correlation: process, file and DNS

The aim was to correlate several Sysmon events that belong to one process. The expected chain was Event 1 (process), Event 11 (file), Event 22 (DNS), Event 3 (network). Command run:

```
powershell.exe -NoProfile -Command "$p='$env:TEMP\DAY04-CORRELATION-TEST.txt'; 'DAY04-
CORRELATION-TEST' | Out-File $p; Resolve-DnsName example.com; curl.exe https://example.com"
```

The command line Sysmon recorded did not keep the `$p` variable as written (most likely the calling shell expanded it first). It is documented exactly as observed:

```
='C:\Users\<user>\AppData\Local\Temp\DAY04-CORRELATION-TEST.txt'; 'DAY04-CORRELATION-TEST'
| Out-File ; Resolve-DnsName example.com; curl.exe https://example.com
```

Observed events, all from the same process:

Event	Time (UTC)	What was recorded	Correlation evidence
1	06:12:11.580	PowerShell process creation, PID 10520, parent PID 1084	ProcessGuid `{28520831-343b-6ac7-d704-000000005e01}`

Event	Time (UTC)	What was recorded	Correlation evidence
11	06:12:12.227	File created: `__PSScriptPolicyTest_3yrc2sv.cdx.ps1` in the user's Temp folder	Same ProcessGuid and PID, 0.647 s after process start
22	06:12:16.696	DNS query for `example.com`, QueryStatus 0 (answers include 172.66.147.243 and 104.20.23.154)	Same ProcessGuid, PID and image
3	not observed	No network connection event	None

The file Event 11 recorded is PowerShell's own policy-test temporary file.

Correlation priority: ProcessGuid is the strongest link. PID, image, user and timestamp are supporting evidence. PID alone is not enough because process IDs can be reused.

Event 3 was not observed. It was not assumed to exist just because the command used `curl.exe`; the documentation records only what the telemetry proves.

Interpretation: process, then file, then DNS from one PowerShell instance would be worth investigating in an unknown environment. Correlation shows that activities are associated, not that they are malicious. Here the activity was intentional, the file was a policy-test file and the DNS query went to `example.com`.

Verdict: Benign controlled activity.



Figure 6.4: Event 1, the PowerShell process (06:12:11.580 UTC, PID 10520).

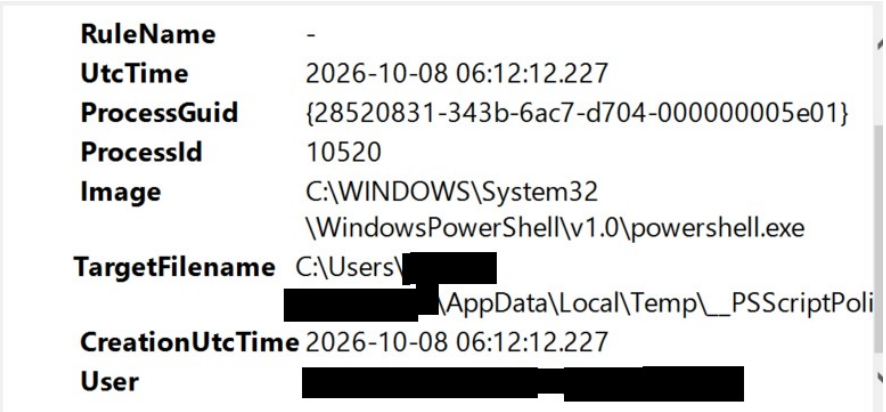


Figure 6.5: Event 11, the temporary file created by the same ProcessGuid (06:12:12.227 UTC).

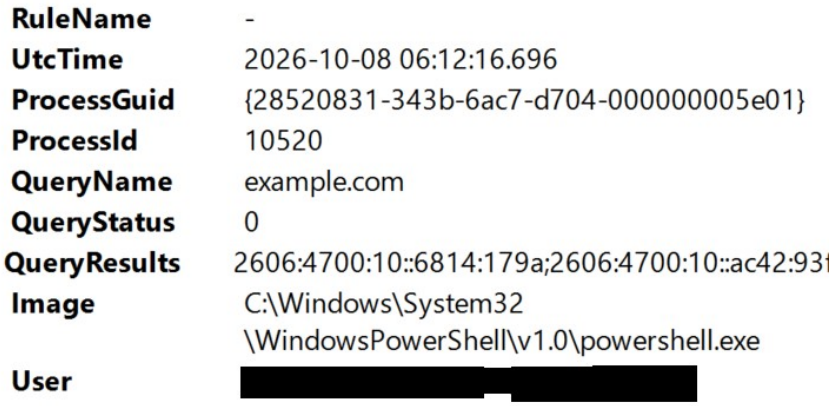


Figure 6.6: Event 22, the example.com DNS query from the same ProcessGuid (06:12:16.696 UTC).

No screenshot exists for Event 3 because it was not observed.

6.5 Validation exercises: blind investigations

Scenario	Evidence	Verdict and reasoning
1	Rule 92027, PowerShell spawning PowerShell, `-NoProfile -Command Get-Date`	**Benign.** The command only returns the date; the alert comes from the parent-child relationship
2	`schtasks /Create /TN "WindowsUpdate"`, then PowerShell `-EncodedCommand`, then an external script download	**Suspicious, investigate immediately.** Three suspicious behaviours are chained; suspicious still does not mean confirmed malware
3	20 failed logons (4625) across multiple accounts from 10.10.20.55 (a private address) in about 2 minutes, then a successful logon (4624)	**Suspicious, investigate.** Brute force or password spraying is the hypothesis, not a confirmed compromise. Check the logon type, which account succeeded and what happened afterwards

Scenario	Evidence	Verdict and reasoning
4	`-EncodedCommand` decoding to `Get-Date`; no Event 11, 22 or 3, no persistence, no suspicious children	**Benign** based on the available evidence and the controlled context

6.6 Phase 4 lessons

- Unusual is not malicious: an odd process chain triggers investigation, not a verdict.
- Telemetry can exist without an alert.
- ProcessGuid is the strongest way to tie Sysmon events to one process instance.
- Correlation shows association, not maliciousness.
- Never invent missing telemetry: write “Event 3 was not observed” instead of completing the expected chain.
- “Suspicious and requires investigation” is a professional verdict; “definitely malware” needs evidence.

7. Phase 5: Windows Authentication Hunting and Authentication Correlation

Phase 5 applies the investigation method to Windows logon events: what failed and successful authentication look like in Wazuh, and why a failure-then-success sequence has to be judged as a whole rather than one alert at a time. The question that drove the phase: **does a sequence of failed logons followed by a successful logon indicate an attack, or can it be legitimate activity?**

7.1 Authentication events

- **Event ID 4625:** an account failed to log on. It is not automatically an attack.
- **Event ID 4624:** an account successfully logged on. It is not automatically legitimate, and not automatically a compromise either.

In Wazuh these appeared as Rule 60122 (Logon Failure - Unknown user or bad password, Level 5) and Rule 60118 (Windows Workstation Logon Success, Level 3). A SOC analyst needs context before giving a verdict on either one.

7.2 Controlled authentication test

A sequence was generated on purpose on the Windows lab machine (agent `WIN10-SOC-LAB`):

```
Wrong password -> 4625 -> Wrong password -> 4625 -> Wrong password -> 4625
-> Correct password -> 4624
```

The events travelled the normal pipeline and were investigated in Wazuh under Threat Hunting, Events:

```
Windows Security Event -> Wazuh Agent -> Wazuh Manager -> Decoder -> Detection Rule ->
Alert -> SOC Investigation
```

7.3 Event 4625 investigation: failed logon

The fields were examined rather than just the alert title:

Field	Value
Message	An account failed to log on
Account	`lab-user` (local account on the lab machine)
Logon type	2 (interactive)
Failure reason	Unknown user name or bad password
Status / Sub status	0xC000006D / 0xC000006A
Source network address / port	127.0.0.1 / 0
Caller process	svchost.exe (PID 0x984, hexadecimal; 2436 in decimal)

A failed logon has many possible explanations: a mistyped password, wrong or expired credentials, an application or service using an old password, or brute force, password spraying and credential abuse. **A single 4625 event is not enough to conclude an attack.**

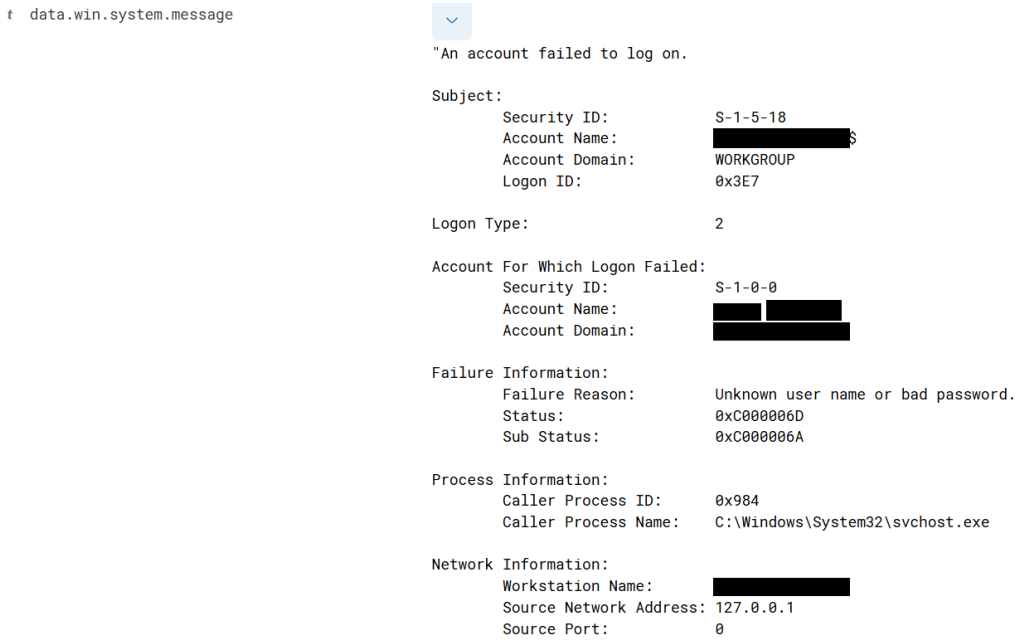


Figure 7.1: Event 4625 in Wazuh: logon type 2, failure reason "Unknown user name or bad password", status 0xC000006D, sub status 0xC000006A, source 127.0.0.1.

7.4 Event 4624 investigation: successful logon

After the correct password was entered, Wazuh captured the matching 4624 event:

Field	Value
Message	An account was successfully logged on
Account	`lab-user` (same account as the failures)
Logon type	2 (interactive)
Elevated token	Yes
Impersonation level	Impersonation
Caller process	svchost.exe (PID 0x984 = 2436, the same as in the failures)

This is important evidence because the success came **after** the failed attempts.

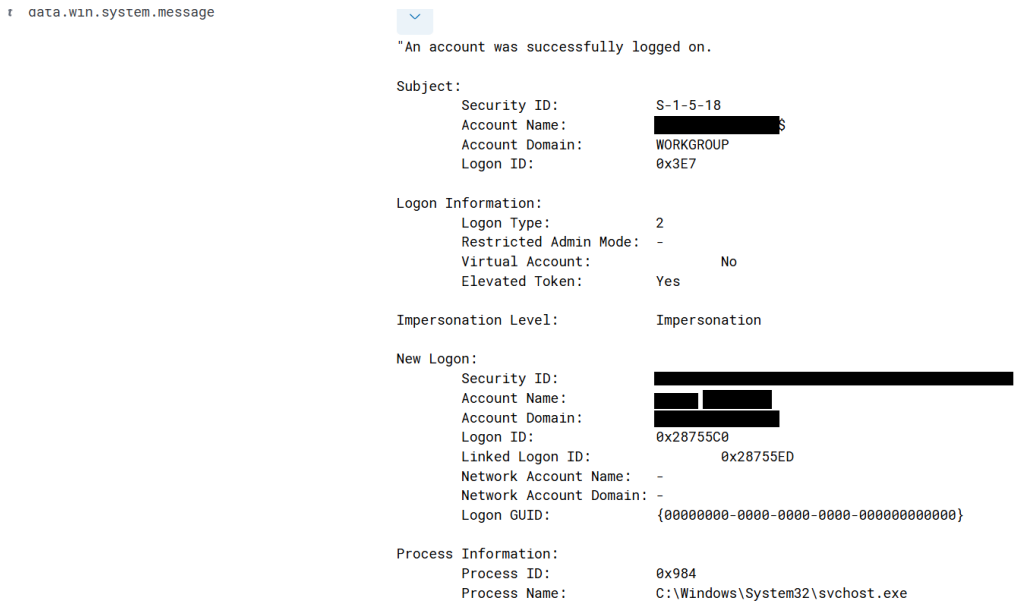


Figure 7.2: Event 4624 in Wazuh: successful interactive logon (type 2) for the same account, elevated token, caller process svchost.exe.

7.5 Authentication correlation

The most important exercise of the phase was looking at the events as one timeline. Wazuh showed them in this order (times as displayed in the dashboard):

Time	Rule	Description	Level	Gap from previous
15:16:35.903	60122	Logon Failure - Unknown user or bad password	5	start
15:16:40.118	60122	Logon Failure - Unknown user or bad password	5	4.2 s
15:16:45.748	60122	Logon Failure - Unknown user or bad password	5	5.6 s
15:16:54.506	60118	Windows Workstation Logon Success	3	8.8 s

Three failures about 5 seconds apart, then one success within 9 seconds: same machine, same account, same logon type and same caller process. That fits a user retyping a password, which is exactly what was done.

```
Multiple failures -> same authentication context -> successful authentication
-> investigate whether the success was expected
```

	Oct 8, 2026 @ 15:16:54.506	WIN10-SOC-LAB	Windows Workstation Logon Success	3	60118
	Oct 8, 2026 @ 15:16:45.748	WIN10-SOC-LAB	Logon Failure - Unknown user or bad password	5	60122
	Oct 8, 2026 @ 15:16:40.118	WIN10-SOC-LAB	Logon Failure - Unknown user or bad password	5	60122
	Oct 8, 2026 @ 15:16:35.903	WIN10-SOC-LAB	Logon Failure - Unknown user or bad password	5	60122

Figure 7.3: Wazuh events in order: three Rule 60122 failures followed by Rule 60118 success.

7.6 What an analyst checks after 4625, 4625, 4625, 4624

- **Account:** which account was targeted? A normal user, an administrator or a service account?
- **Source:** where did the logon come from, and is that source internal and expected for this user?
- **Logon type:** type 2 is interactive, type 3 is network, type 10 is remote interactive (RDP). The type changes how the sequence reads.
- **Timing:** how many failures, how far apart, and is that normal for this environment?
- **Post-authentication activity:** PowerShell or cmd, new processes, scheduled tasks or services, file creation, network connections, privilege escalation, lateral movement. This links authentication back to the process and correlation work of Phases 3 and 4.

7.7 Same pattern, different meaning

Example	Pattern	Assessment
A	4625 x3, then 4624	**Benign.** The user forgot the password and then typed the right one
B	4625 x20, multiple accounts, same source IP, then 4624	**Suspicious.** Possible password spraying (compare the Phase 3 blind scenario with 27 failures from 10.10.20.55)
C	4625 x many against a privileged account, 4624, then PowerShell and network activity	**Highly suspicious.** Possible account compromise needing immediate investigation

The event itself is only one piece of evidence. The sequence and the context set the investigation priority.

7.8 Verdict and workflow

Verdict: Benign, controlled authentication testing. The failures were deliberate wrong-password attempts followed by the legitimate password. This activity alone shows no credential compromise, brute force, password spraying or account takeover.

The workflow developed in this phase:

```
Authentication alert -> event ID -> account -> source -> logon type -> timestamp
-> count failures -> look for a successful logon -> correlate events
-> investigate post-login activity -> assess context -> verdict
```

Lessons:

- 4625 is a failed authentication, not automatically an attack; 4624 is a successful one, not automatically legitimate.
- Logon type provides important context.

- Multiple failures mean more as a pattern, and a success after failures deserves more investigation.
- Post-authentication activity shows whether an account was actually abused.
- Correlation beats isolated alerts, and context decides the verdict.

8. Phase 6: Network Security Monitoring and Custom Rule Troubleshooting

Phase 6 turns to network activity from a SOC analyst's view: reading Sysmon Event ID 3, correlating process, DNS, network and file events, and troubleshooting a custom network rule. This phase is documented with its failure included: a custom rule that never produced an alert is recorded as **unresolved**. Environment: Wazuh 4.14.7, Windows 10 Pro endpoint `WIN10-SOC-LAB` (Agent ID 001), manager `wazuh-server` and Sysmon.

Core principle: an alert or suspicious event is a starting point for investigation, not proof of compromise.

8.1 Tools and their separate jobs

- **Sysmon** records configured endpoint activity.
- **Wazuh** receives and processes endpoint events, evaluates rules and alerts when a rule matches.

8.2 Sysmon Event ID 3 (network connection)

Event 3 is only recorded when network connection monitoring is enabled in the Sysmon configuration.

Field	Investigation purpose
`Image` / `ProcessId` / `ProcessGuid`	Which process made the connection, and the key for correlation
`User`	Account that owns the process
`Protocol` / `Initiated`	Transport protocol, and whether the process started the connection
`SourceIp` / `SourcePort`	Local end of the connection
`DestinationIp` / `DestinationPort`	Remote end of the connection
`UtcTime`	When it happened

Why port 443 is not a verdict. TCP 443 is normally HTTPS/TLS, but a connection to it does not show that the traffic is legitimate or malicious. The analyst asks: which process started it, what command line launched that process, which destination IP and domain were involved, was there a DNS query first, does it fit the timeline, and is there evidence of a download, execution or persistence.

8.3 Correlating Sysmon events

Event	Meaning	Use
1	Process creation	Image, command line, parent, user, PID and ProcessGuid
3	Network connection	Network activity tied to a process
11	File creation	Files a process created
22	DNS query	Domains a process asked about

Method: find the process-creation event, note its image, command line, user, PID and ProcessGuid, find network and DNS events with the same identifiers and compatible times, check file activity, rebuild the timeline, then state what the evidence supports and what stays unknown. A matching ProcessGuid is strong evidence of the same process. A similar timestamp alone does not prove causation, and a child process has its own ProcessGuid, so parent-child links must be checked separately.

8.4 Endpoint evidence from Event Viewer (8 October 2026)

These screenshots come from the Sysmon event details on the endpoint (the Friendly View of Windows Event Viewer) and are timestamped 8 October 2026 (UTC). They are separate from the Wazuh archive records in Section 8.5, which are dated 9 October.

nslookup: process creation and network connection. `nslookup example.com` was run from PowerShell. Sysmon recorded the process creation (Event 1) and a network connection (Event 3) for `nslookup.exe`.

Event	What was recorded
1	`nslookup.exe`, command line `nslookup example.com`, user `lab-user`, Medium integrity, parent `powershell.exe` (PID 1084)
3	`nslookup.exe` (PID 12476), 2026-10-08 11:21:02.833 UTC, protocol **udp**, initiated **true**, ProcessGuid `{28520831-7c9e-6ac7-f609-000000005e01}`, source [endpoint IP]:50615, destination **8.8.8.8** (`dns.google`) port **53** (domain)

This is a clear example of Event 3 being recorded in the endpoint log: the DNS client sent a UDP query to a public resolver on port 53, which is the standard DNS port and fits the command. The Event 1 screenshots below do not show that record's ProcessGuid or timestamp, so the formal GUID link between the Event 1 and the Event 3 should be confirmed before it is stated as proven.

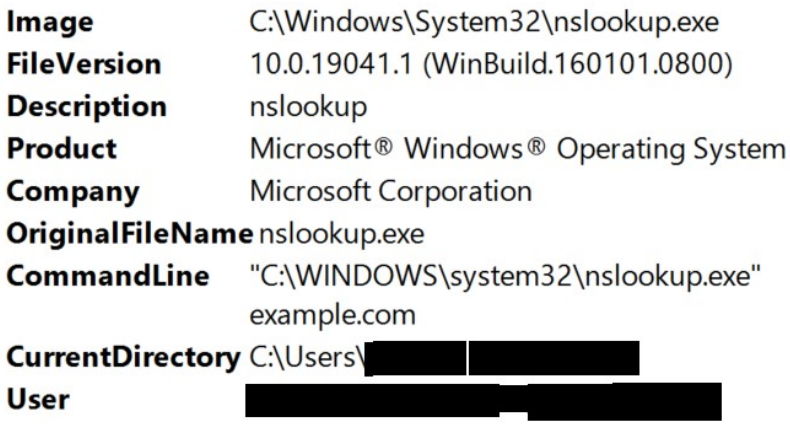


Figure 8.1: Sysmon Event 1 for nslookup.exe (upper part): command line nslookup example.com, Microsoft Corporation.

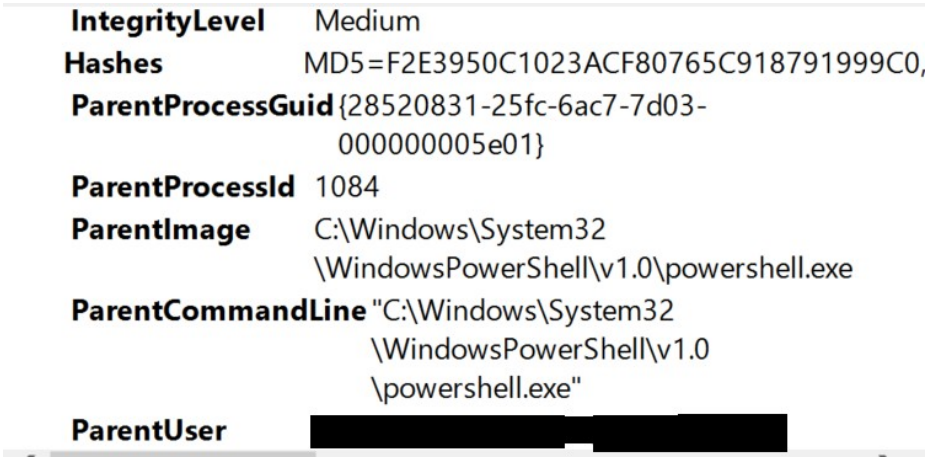


Figure 8.2: Sysmon Event 1 for nslookup.exe (lower part): Medium integrity, parent PowerShell (PID 1084).

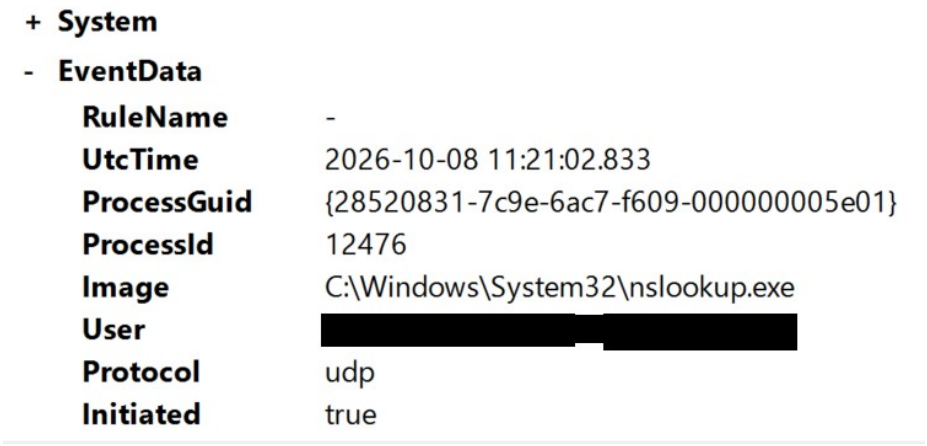


Figure 8.3: Sysmon Event 3 for nslookup.exe (upper part): UDP, initiated true, PID 12476, 2026-10-08 11:21:02.833 UTC.

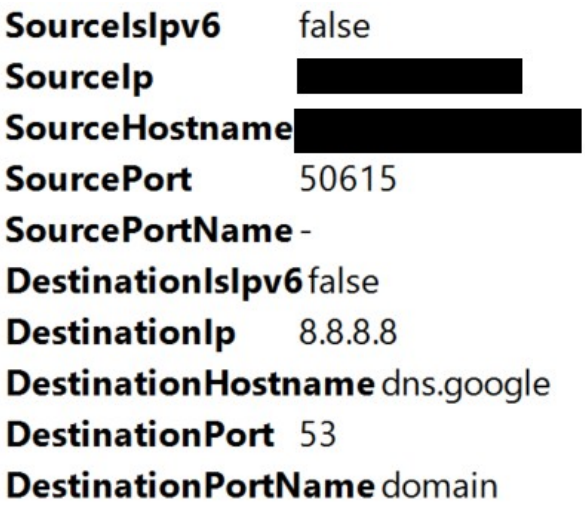


Figure 8.4: Sysmon Event 3 for nslookup.exe (lower part): source [endpoint IP]:50615, destination 8.8.8.8 (dns.google) port 53 (domain).

curl.exe process creation. Sysmon also recorded the process creation of `curl.exe` for a test request to `https://www.microsoft.com`. No Event 3 screenshot for this request is included, so nothing is claimed here about its network connection.

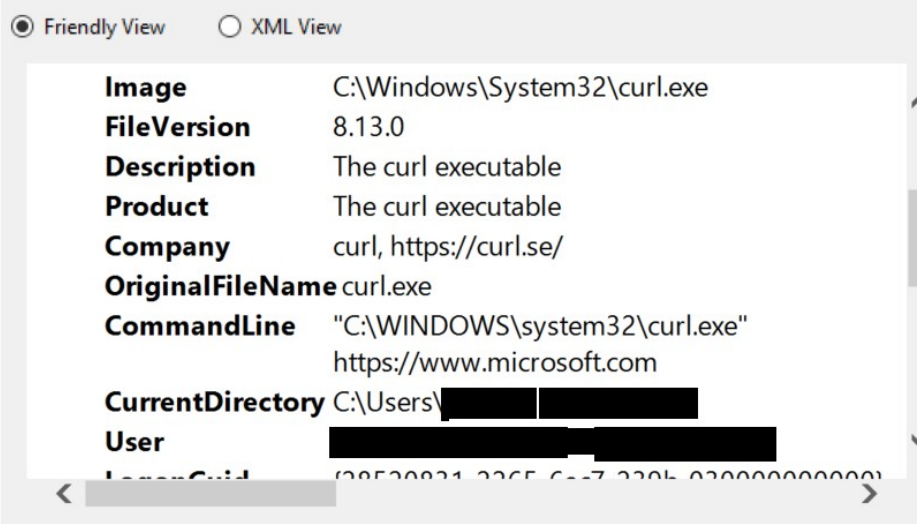


Figure 8.5: Sysmon Event 1 for curl.exe 8.13.0, command line curl.exe https://www.microsoft.com.

PowerShell DNS query. An Event 22 from a PowerShell process asking for `example.com`.

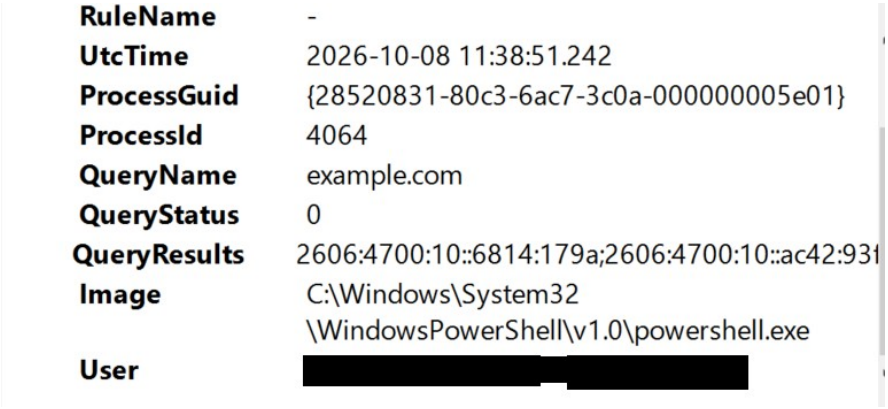


Figure 8.6: Sysmon Event 22: PowerShell (PID 4064) queried example.com, QueryStatus 0, 2026-10-08 11:38:51.242 UTC.

8.5 Wazuh archive evidence: Sysmon Event 3 (9 October 2026)

While troubleshooting, a real Event 3 was found in the manager's JSON archive. This is real lab evidence, not a hypothetical.

Field	Observed value
Archive timestamp	2026-10-09T12:26:58.636+0000 (the event's own UTC time was 12:26:57.021; these are different fields)
Agent / manager	`WIN10-SOC-LAB` (ID 001) / `wazuh-server`
Provider / Event ID / decoder	Microsoft-Windows-Sysmon / 3 / `windows_eventchannel`

Field	Observed value
Source	[endpoint IP]:59864
Destination	104.20.23.154:443
Protocol / Initiated	tcp / true
Process	PowerShell, PID 17776, ProcessGuid `{28520831-cdf7-6ac8-9b06-000000005f01}`
User	`lab-user`

What it established: an Event 3 reached the manager's archive, it was tied to agent 001 and the `windows_eventchannel` decoder, the process was PowerShell, and it carried TCP metadata for port 443.

What it did not establish: that the destination was malicious, that the event produced an alert, that custom rule 100103 works, or what data was sent over the connection.

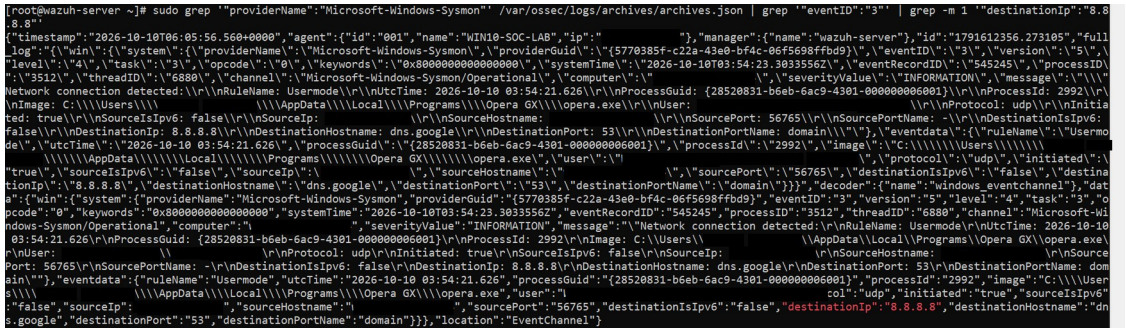

[root@wazuh-server ~]# sudo grep '"providerName":"Microsoft-Windows-Sysmon"' /var/ossec/logs/archives/archives.json | grep '"eventID":"3"' | grep -m 1 '"destinationIp":"8.8.8.8"'
{"timestamp":"2026-10-10T06:05:56.560+0000","agent":{"id":"001","name":"WIN10-SOC-LAB","ip":"[redacted]"},"manager":{"name":"wazuh-server"},"id":"1791612356.273105","full_log":"{\"win\":{\"system\":{\"providerName\":\"Microsoft-Windows-Sysmon\",\"providerGuid\":\"{5770385f-c22a-43e0-bf4c-06f5698ffbd9}\",\"eventID\":\"3\",\"version\":\"5\",\"level\":\"4\",\"task\":\"3\",\"opcode\":\"0\",\"keywords\":\"0x8000000000000000\",\"systemTime\":\"2026-10-10T03:54:23.3033556Z\",\"eventRecordID\":\"545245\",\"processID\":\"3512\",\"threadID\":\"6880\",\"channel\":\"Microsoft-Windows-Sysmon/Operational\",\"computer\":\"[redacted]\",\"severityValue\":\"INFORMATION\",\"message\":\"\\\"Network connection detected:\\r\\nRuleName: Usermode\\r\\nUtcTime: 2026-10-10 03:54:21.626\\r\\nProcessGuid: {28520831-b6eb-6ac9-4301-000000006001}\\r\\nProcessId: 2992\\r\\nImage: C:\\\\Users\\\\[redacted]\\\\AppData\\\\Local\\\\Programs\\\\Opera GX\\\\opera.exe\\r\\nUser: [redacted]\\r\\nProtocol: udp\\r\\nInitiated: true\\r\\nSourceIsIpv6: false\\r\\nSourceIp: [redacted]\\r\\nSourceHostname: [redacted]\\r\\nSourcePort: 56765\\r\\nSourcePortName: -\\r\\nDestinationIsIpv6: false\\r\\nDestinationIp: 8.8.8.8\\r\\nDestinationHostname: dns.google\\r\\nDestinationPort: 53\\r\\nDestinationPortName: domain\\\"\"},\"eventdata\":{\"ruleName\":\"Usermode\",\"utcTime\":\"2026-10-10 03:54:21.626\",\"processGuid\":\"{28520831-b6eb-6ac9-4301-000000006001}\",\"processId\":\"2992\",\"image\":\"C:\\\\Users\\\\[redacted]\\\\AppData\\\\Local\\\\Programs\\\\Opera GX\\\\opera.exe\",\"user\":\"[redacted]\",\"protocol\":\"udp\",\"initiated\":\"true\",\"sourceIsIpv6\":\"false\",\"sourceIp\":\"[redacted]\",\"sourceHostname\":\"[redacted]\",\"sourcePort\":\"56765\",\"destinationIsIpv6\":\"false\",\"destinationIp\":\"8.8.8.8\",\"destinationHostname\":\"dns.google\",\"destinationPort\":\"53\",\"destinationPortName\":\"domain\"}}}","decoder":{"name":"windows_eventchannel"},"data":{"win":{"system":{"providerName":"Microsoft-Windows-Sysmon","providerGuid":"{5770385f-c22a-43e0-bf4c-06f5698ffbd9}","eventID":"3","version":"5","level":"4","task":"3","opcode":"0","keywords":"0x8000000000000000","systemTime":"2026-10-10T03:54:23.3033556Z","eventRecordID":"545245","processID":"3512","threadID":"6880","channel":"Microsoft-Windows-Sysmon/Operational","computer":"[redacted]","severityValue":"INFORMATION","message":"\"Network connection detected:\r\nRuleName: Usermode\r\nUtcTime: 2026-10-10 03:54:21.626\r\nProcessGuid: {28520831-b6eb-6ac9-4301-000000006001}\r\nProcessId: 2992\r\nImage: C:\\Users\\[redacted]\\AppData\\Local\\Programs\\Opera GX\\opera.exe\r\nUser: [redacted]\r\nProtocol: udp\r\nInitiated: true\r\nSourceIsIpv6: false\r\nSourceIp: [redacted]\r\nSourceHostname: [redacted]\r\nSourcePort: 56765\r\nSourcePortName: -\r\nDestinationIsIpv6: false\r\nDestinationIp: 8.8.8.8\r\nDestinationHostname: dns.google\r\nDestinationPort: 53\r\nDestinationPortName: domain\"","eventdata":{"ruleName":"Usermode","utcTime":"2026-10-10 03:54:21.626","processGuid":"{28520831-b6eb-6ac9-4301-000000006001}","processId":"2992","image":"C:\\\\Users\\\\[redacted]\\\\AppData\\\\Local\\\\Programs\\\\Opera GX\\\\opera.exe","user":"[redacted]","protocol":"udp","initiated":"true","sourceIsIpv6":"false","sourceIp":"[redacted]","sourceHostname":"[redacted]","sourcePort":"56765","destinationIsIpv6":"false","destinationIp":"8.8.8.8","destinationHostname":"dns.google","destinationPort":"53","destinationPortName":"domain"}}},"location":"EventChannel"}

Figure 8.7: A second archived Sysmon Event 3 (10 October 2026): Opera GX sending UDP traffic to 8.8.8.8:53, decoded by windows_eventchannel. It proves the archive and decoder path for Event 3; it is not the PowerShell record described above. Names are blacked out for publication.

This screenshot shows a different Event 3 from the table above (Opera GX, UDP, 10 October), and it is discussed in Phase 7 (Section 9.2).

8.6 Wazuh archive evidence: matching DNS (Event 22)

An archived Event 22 carried the same ProcessGuid `{28520831-cdf7-6ac8-9b06-000000005f01}` (PID 17776, `powershell.exe`) with query name `example.com`, and one recorded result included `104.20.23.154`. That supports linking the DNS lookup to the PowerShell connection to `104.20.23.154:443` by ProcessGuid. A further Event 22 near 2026-10-09 12:26:57.374 had DNS status 9701, and another nearby record had status 0 with IPv6 results plus `104.20.23.154`. These are separate records and are not merged.

This is endpoint-event correlation. It does not by itself show malicious activity.

8.7 Troubleshooting custom rule 100103 (unresolved)

The problem. Event 3 records were in the manager's JSON archive, but the custom rule meant to detect PowerShell network connections never produced an alert: no alert with ID `100103` was found in the alerts file.

Built-in rule. Rule 61605 in `/var/ossec/ruleset/rules/0595-win-sysmon_rules.xml` matches Sysmon Event 3. Its level is 0, so it works as an informational parent and does not alert by itself:

```
<rule id="61605" level="0">
  <if_sid>61600</if_sid>
  <field name="win.system.eventID">^3$</field>
  <description>Sysmon - Event 3: Network connection to $(win.eventdata.destinationIp):$
(win.eventdata.destinationPort) by $(win.eventdata.image)</description>
  <options>no_full_log</options>
  <group>sysmon_event3,</group>
</rule>
```

Custom rule. Rule 100103 lives in `/var/ossec/etc/rules/local_rules.xml`, inside the outer group of that file (shown as `local,syslog,sshd,` in the Phase 8 capture, Section 10.8), and uses a group condition:

```
<rule id="100103" level="8">
  <if_group>sysmon_event3</if_group>
  <field name="win.eventdata.image" type="pcre2">(?i)\\powershell\.exe$</field>
  <field name="win.eventdata.protocol">^tcp$</field>
  <description>powershell initiated a TCP network connection to $
(win.eventdata.destinationIp):$(win.eventdata.destinationPort)</description>
  <group>sysmon_event3,powershell,network_connection,</group>
</rule>
```

```
<rule id="100103" level="8">
    <if_group>sysmon_event3</if_group>
    <field name="win.eventdata.image" type="pcre2">(?i)\\powershell\.exe$</field>
    <field name="win.eventdata.protocol">^tcp$</field>
    <description>powershell initiated a TCP network connection to $(win.eventdata.destinationIp):$(win.eventdata.destinationPort
)</description>
    <group>sysmon_event3,powershell,network_connection,</group>
    </rule>
</group>
```

Figure 8.8: Rule 100103 in local_rules.xml: if_group sysmon_event3, image ends in powershell.exe, protocol tcp, level 8.

Steps and what each one showed

1. **Validation.** `sudo /var/ossec/bin/wazuh-analysisd -t` reported no configuration errors. That shows the files are valid, not that the rule matches a real Event 3.
2. **Searching for the alert.** `grep '"id":"100103"' /var/ossec/logs/alerts/alerts.json` found nothing. This shows the file had no such alert at that time, not why.
3. **Turning on JSON archiving.** To see incoming events, `<logall_json>yes</logall_json>` was set temporarily (with `<logall>no</logall>` unchanged) and `/var/ossec/logs/archives/archives.json` was checked. A real Event 3 appeared there, with `decoder.name` `windows_eventchannel` and `location` `EventChannel`. So the event reached the manager and was represented as an EventChannel record, yet the alert was still missing.
4. **wazuh-logtest.** A manual test returned `Phase 2: No decoder matched`. The input had not been rebuilt in a form the decoder accepts, so this does **not** prove the live event was undecoded. A later attempt to pull an event out of the archive returned `NO_MATCHING_EVENT` because the filter did not fit the escaped, nested JSON, and one Python attempt hit a variable-name `NameError` that was then fixed.

The pipeline stages that were separated: Sysmon creates the event, Wazuh receives it, the decoder parses it, rules are evaluated, and a matching alert-producing rule raises the alert. The failure sits somewhere after “decoder identified” and before “alert raised”.

Possible lead. Wazuh GitHub issue #38097 describes similar reports of Sysmon Events 3 and 22 being archived without the expected alerts in some versions. The lab runs 4.14.7. It is a research lead only; similar symptoms do not prove the same cause.

Final status: unresolved, further troubleshooting deferred.

Established	Not established
A real Event 3 exists in the archive	Why rule 100103 produced no alert
The archived record shows the `windows_eventchannel` decoder	Any working fix
Built-in rule 61605 exists and 100103 is in the reviewed config	That issue #38097 explains this behaviour
Configuration validation reported no errors	
No alert with ID 100103 was found	
Manual logtest attempts did not reproduce the archived event	

No successful fix is claimed.

Cleanup to check. JSON archiving was enabled only to diagnose the problem, and its final state was **not verified**. Check `/var/ossec/etc/ossec.conf`. If `<logall_json>` is still `yes` and is no longer needed, set it back to `no`, keep `<logall>no</logall>`, validate, restart the manager if required and confirm the service is running. Leaving it on indefinitely can use a lot of disk space.

8.8 Hypothetical investigation: Word launching PowerShell

> **Training scenario only.** This was a fictional exercise. It was not an attack on the lab endpoint, and none of its evidence comes from the lab.

Stage	Evidence in the scenario	Reading
1	`WINWORD.EXE` starts `powershell.exe` and an outbound TCP 443 connection follows	**Suspicious, needs investigation.** Word launching PowerShell can mean a malicious document, but also unusual legitimate workflows
2	PowerShell uses `-NoProfile`, `-WindowStyle Hidden` and `-EncodedCommand`; payload unknown	Concern rises in combination, but encoding alone and hidden execution alone prove nothing
3	PowerShell tries to download a script, a `.ps1` file appears in the temp folder (Event 11), a network connection occurs; no proof of execution or persistence	Still not confirmed

Next steps: decode the command, examine the script safely, record its hash and signature, correlate process, file, DNS and network records, look for execution, check scheduled tasks and registry if persistence is

suspected, and establish whether the activity was authorized. Chain to assess as a whole: `WINWORD.EXE` then `powershell.exe`, outbound connection, script file created, then execution and impact. **Final scenario verdict: suspicious, pending investigation.** The evidence does not support claiming malware or a real compromise.

8.9 Hypothetical network exercises

> These are also training scenarios, not lab events.

Exercise	Scenario	Verdict and next steps
A	Encoded PowerShell queries an unfamiliar domain, the same ProcessGuid connects to an external IP on 443, and a file is created in temp	**Suspicious.** The chain warrants investigation but does not prove the file is malicious. Decode the command, examine the destination, inspect the file safely, establish whether it ran
B	A decoded command downloads a file from an unfamiliar server into temp	**Suspicious, immediate investigation reasonable.** Preserve evidence, check hash and signature, determine whether it ran. A download alone is not proof of malware
C	One workstation makes 30 connection attempts to 25 internal IPs in 20 seconds, started by PowerShell, no approved scan documented	**Suspicious.** Could be discovery or port scanning, but authorized scanning looks similar. Check the command line, ports and outcomes, parent-child chain, and any approved scanning
D	Several connections to internal systems on TCP 445, then a new process with unknown purpose	**Suspicious, pending.** Examine the new process (Event 1, parent, command line, user, privileges, file and network activity) and whether it relates to the earlier scanning. Port 445 is SMB, but the port alone proves nothing
E	PowerShell launches `cmd.exe` running `whoami` and `ipconfig` as the logged-in user	**Needs further investigation**, with the earlier scanning kept in view. Both commands are legitimate; judge the combined evidence, not one command

8.10 Verdict framework

Verdict	Meaning
Benign	Evidence supports legitimate activity in context
Suspicious	Concerning characteristics that need further investigation
Needs further investigation	Not enough evidence for a reliable classification
Confirmed malicious	Evidence sufficiently establishes malicious activity

A suspicious event is not automatically malware, and the absence of an alert does not mean nothing happened.

8.11 Phase 6 lessons

- Event collection is not alert generation: an event can sit in the archive without the expected custom alert.
- Decoder success is not rule-match success.
- Configuration validation does not prove detection logic works.
- ProcessGuids support correlation; timestamps and parent-child links add context.
- Network ports are not verdicts: 443 and 445 need destination, process, intent and surrounding activity.
- A correlated suspicious sequence deserves investigation, but still needs validation.
- Hypothetical exercises must stay labelled as hypothetical.
- An unresolved issue must be documented honestly: a troubleshooting attempt is not a fix.

8.12 Phase 6 status

Covered: Event 3 fields, Events 1/3/11/22 correlation concepts, Wazuh archive, decoder, rule and alert stages, the troubleshooting history of rule 100103, and the hypothetical investigations.

Deferred or unresolved:

1. The root cause of the missing alert for rule 100103.
2. Whether Wazuh issue #38097 applies to this lab.
3. The final state of temporary JSON archiving.

9. Phase 7: MITRE ATT&CK Mapping and Detection Validation

Phase 7 asks two questions that every detection engineer has to answer honestly. First: **can an observed event really be mapped to a MITRE ATT&CK technique?** Second: **did the Wazuh detection pipeline actually produce the alert I expected?** The evidence for this phase was collected on 10 October 2026 and reviewed on 11 October 2026.

9.1 Phase 7 at a glance

Question	Evidence	Answer
Does Wazuh collect Sysmon network telemetry?	Archived Sysmon Event 3 record (Figure 9.1)	**Yes**, collected, archived and decoded by `windows_eventchannel`
Was an alert generated for rule 61605 or 100103?	`grep` of `alerts.json` for both IDs	**No matching record found**; see Section 9.3 for how to read this
Is the event malicious?	`opera.exe` to `8.8.8.8:53` over UDP	**No malicious activity established**; consistent with ordinary DNS
Can it be mapped to ATT&CK?	Single event, no adversarial context	**No technique confirmed**; T1071.004 is not assigned

Environment: Wazuh 4.14.7, manager `wazuh-server`, agent `WIN10-SOC-LAB` (ID 001), Sysmon telemetry from `Microsoft-Windows-Sysmon/Operational`.

9.2 Practical 1: verify Sysmon network telemetry

A real Sysmon Event ID 3 record was located in the manager's archive with this command (first Event 3 whose destination is 8.8.8.8):

```
sudo grep '"providerName":"Microsoft-Windows-Sysmon"'
/var/ossec/logs/archives/archives.json \
  | grep '"eventID":"3"' | grep -m 1 '"destinationIp":"8.8.8.8"'
```

Field	Observed value
Sysmon Event ID	3 (network connection)
Process	`opera.exe` (Opera GX), PID 2992
ProcessGuid	`{28520831-b6eb-6ac9-4301-000000006001}`
Protocol	UDP, initiated true
Source	[endpoint IP], port 56765
Destination	8.8.8.8 (`dns.google`), port 53 (domain)
Event time (UTC)	2026-10-10 03:54:21.626
Archive timestamp	2026-10-10T06:05:56.560+0000
Decoder	`windows_eventchannel`

Field	Observed value
Agent / manager	`WIN10-SOC-LAB` (ID 001) / `wazuh-server`

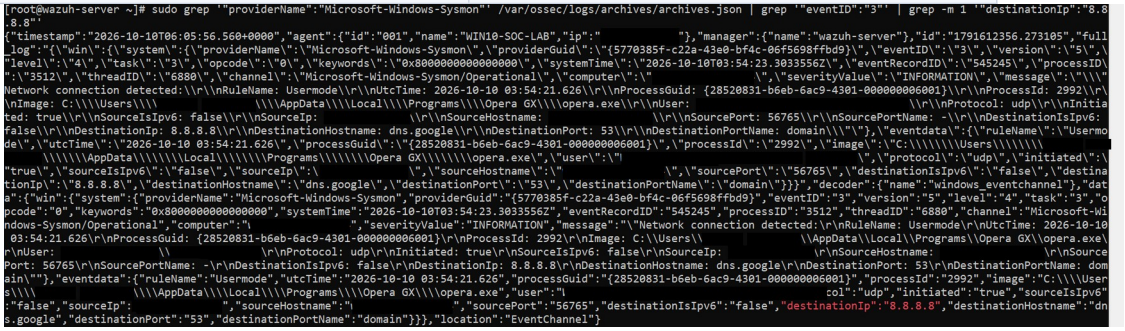

[root@wazuh-server ~]# sudo grep '"providerName":"Microsoft-Windows-Sysmon"' /var/ossec/logs/archives/archives.json | grep '"eventID":"3"' | grep -m 1 '"destinationIp":"8.8.8.8"'
{"timestamp":"2026-10-10T06:05:56.560+0000","agent":{"id":"001","name":"WIN10-SOC-LAB","ip":"[redacted]"},"manager":{"name":"wazuh-server"},"id":"1791612356.273105","full_log":"{\"win\":{\"system\":{\"providerName\":\"Microsoft-Windows-Sysmon\",\"providerGuid\":\"{5770385f-c22a-43e0-bf4c-06f5698ffbd9}\",\"eventID\":\"3\",\"version\":\"5\",\"level\":\"4\",\"task\":\"3\",\"opcode\":\"0\",\"keywords\":\"0x8000000000000000\",\"systemTime\":\"2026-10-10T03:54:23.3033556Z\",\"eventRecordID\":\"545245\",\"processID\":\"3512\",\"threadID\":\"6880\",\"channel\":\"Microsoft-Windows-Sysmon/Operational\",\"computer\":\"[redacted]\",\"severityValue\":\"INFORMATION\",\"message\":\"\\\"Network connection detected:\\r\\nRuleName: Usermode\\r\\nUtcTime: 2026-10-10 03:54:21.626\\r\\nProcessGuid: {28520831-b6eb-6ac9-4301-000000006001}\\r\\nProcessId: 2992\\r\\nImage: C:\\\\Users\\\\[redacted]\\\\AppData\\\\Local\\\\Programs\\\\Opera GX\\\\opera.exe\\r\\nUser: [redacted]\\r\\nProtocol: udp\\r\\nInitiated: true\\r\\nSourceIsIpv6: false\\r\\nSourceIp: [redacted]\\r\\nSourceHostname: [redacted]\\r\\nSourcePort: 56765\\r\\nSourcePortName: -\\r\\nDestinationIsIpv6: false\\r\\nDestinationIp: 8.8.8.8\\r\\nDestinationHostname: dns.google\\r\\nDestinationPort: 53\\r\\nDestinationPortName: domain\\\"\"},\"eventdata\":{\"ruleName\":\"Usermode\",\"utcTime\":\"2026-10-10 03:54:21.626\",\"processGuid\":\"{28520831-b6eb-6ac9-4301-000000006001}\",\"processId\":\"2992\",\"image\":\"C:\\\\Users\\\\[redacted]\\\\AppData\\\\Local\\\\Programs\\\\Opera GX\\\\opera.exe\",\"user\":\"[redacted]\",\"protocol\":\"udp\",\"initiated\":\"true\",\"sourceIsIpv6\":\"false\",\"sourceIp\":\"[redacted]\",\"sourceHostname\":\"[redacted]\",\"sourcePort\":\"56765\",\"destinationIsIpv6\":\"false\",\"destinationIp\":\"8.8.8.8\",\"destinationHostname\":\"dns.google\",\"destinationPort\":\"53\",\"destinationPortName\":\"domain\"}}}","decoder":{"name":"windows_eventchannel"},"data":{"win":{"system":{"providerName":"Microsoft-Windows-Sysmon","providerGuid":"{5770385f-c22a-43e0-bf4c-06f5698ffbd9}","eventID":"3","version":"5","level":"4","task":"3","opcode":"0","keywords":"0x8000000000000000","systemTime":"2026-10-10T03:54:23.3033556Z","eventRecordID":"545245","processID":"3512","threadID":"6880","channel":"Microsoft-Windows-Sysmon/Operational","computer":"[redacted]","severityValue":"INFORMATION","message":"\"Network connection detected:\r\nRuleName: Usermode\r\nUtcTime: 2026-10-10 03:54:21.626\r\nProcessGuid: {28520831-b6eb-6ac9-4301-000000006001}\r\nProcessId: 2992\r\nImage: C:\\Users\\[redacted]\\AppData\\Local\\Programs\\Opera GX\\opera.exe\r\nUser: [redacted]\r\nProtocol: udp\r\nInitiated: true\r\nSourceIsIpv6: false\r\nSourceIp: [redacted]\r\nSourceHostname: [redacted]\r\nSourcePort: 56765\r\nSourcePortName: -\r\nDestinationIsIpv6: false\r\nDestinationIp: 8.8.8.8\r\nDestinationHostname: dns.google\r\nDestinationPort: 53\r\nDestinationPortName: domain\""},"eventdata":{"ruleName":"Usermode","utcTime":"2026-10-10 03:54:21.626","processGuid":"{28520831-b6eb-6ac9-4301-000000006001}","processId":"2992","image":"C:\\\\Users\\[redacted]\\AppData\\Local\\Programs\\Opera GX\\opera.exe","user":"[redacted]col":"udp","initiated":"true","sourceIsIpv6":"false","sourceIp":"[redacted]","sourceHostname":"[redacted]","sourcePort":"56765","destinationIsIpv6":"false","destinationIp":"8.8.8.8","destinationHostname":"dns.google","destinationPort":"53","destinationPortName":"domain"}}},"location":"EventChannel"}

Figure 9.1: Archived Sysmon Event 3 in /var/ossec/logs/archives/archives.json: Opera GX (opera.exe, PID 2992) sent UDP traffic from [endpoint IP]:56765 to 8.8.8.8:53 (dns.google), decoded by windows_eventchannel. Machine and user names are blacked out for publication.

> **Key takeaway:** Sysmon network telemetry reaches the Wazuh manager and is decoded correctly. Collection and decoding work for Event 3.

A detail worth noting. The archive timestamp (06:05:56) is about **2 hours 11 minutes after** the event's own UTC time (03:54:21). In Phase 6 the same two fields differed by under two seconds. Phase 8 found a boot-time Event 1 archived eight seconds earlier (06:05:48) with the same delay (Figure 10.2), which suggests one backlog of events delivered together after the agent had been unable to reach the manager. That is a hypothesis, not a confirmed cause.

9.3 Practical 2: check alert generation

Two searches were run against the alerts file:

```
sudo grep '"id":"61605"' /var/ossec/logs/alerts/alerts.json | tail -n 5
sudo grep '"id":"100103"' /var/ossec/logs/alerts/alerts.json | tail -n 5
```

Both returned **no matching records**. The original conclusion was that the expected detection was not verified and that the cause was unresolved.



Figure 9.2: Where the Phase 7 evidence stops in the Wazuh pipeline. The event was collected, archived and decoded; no alert record was found.

Reading the result correctly (added during documentation review). These two searches do not prove that the detection pipeline failed for this event, for two reasons that follow from the rules themselves (Section 8.7 and Section 14):

1. **Rule 61605 has level 0.** It is an informational parent rule, and by design it does not write alerts. The alerts file normally holds only alerts at or above the configured alert level (`log_alert_level` in `ossec.conf`), so an empty search for 61605 is the expected result.
2. **Rule 100103 could not match this event.** It requires `powershell.exe` and protocol `tcp`. This record is `opera.exe` over UDP, so the rule is correct to stay silent.

> **Key takeaway:** the empty searches show that an archived event is not an alert. They do not test rule 100103. A valid test needs a **PowerShell TCP** Event 3, such as the 9 October record in Section 8.5, and a search for `100103` after it. That open problem from Phase 6 remains unresolved.

9.4 Practical 3: interpret the event

Opera GX contacted Google's public DNS resolver on UDP port 53. A browser resolving names through a public resolver is ordinary behaviour, and a network connection alone does not establish malicious activity. Facts that would change the picture, none of which were observed: an unusual process making the connection, a rare or unexpected DNS server, very high query volume or very long query names, or a connection from a process that should not use the network.

9.5 MITRE ATT&CK mapping method

MITRE ATT&CK mapping has to be supported by observed behaviour. An event ID, a tool name or a rule level is not enough on its own. The method used:

```
Observed behaviour -> candidate technique -> evidence that would confirm it
-> context (is it authorised or normal?) -> mapping decision
```

Observed behaviour	Candidate technique	Evidence needed to confirm	Decision
`schtasks /Create ... /SC ONLOGON` (Phase 3)	T1053.005 Scheduled Task	Command line, what the task runs, who created it	**Mapped** in rule 100102 as a detection tag; the lab test itself was benign
`-EncodedCommand` PowerShell (Phase 4)	T1059.001, T1027	Decoded content, parent process, follow-on activity	**Candidate only**; the content decoded to `Get-Date`
Three failed logons, then success (Phase 5)	T1110 Brute Force	Many attempts, several accounts, an external source	**Not mapped**; one local account, three attempts, controlled test
`opera.exe` UDP to 8.8.8.8:53 (Phase 7)	T1071.004 DNS	Unusual resolver, volume or query pattern, adversarial context	**Not mapped**; consistent with ordinary DNS

> **Key takeaway:** a MITRE tag describes a technique. It is a hypothesis to test, not proof that an attacker used it.

9.6 Verdict and status

> **Verdict:** No malicious activity established from the available evidence. This applies only to the evidence reviewed and does not prove the endpoint is completely safe.

Item	Status
Sysmon Event 3 collected, archived and decoded	Verified
Event consistent with ordinary DNS	Assessed, benign
ATT&CK technique for this event	None confirmed, none assigned

Item	Status
Alert for rule 61605	Not expected (level 0), no record found
Alert for rule 100103 on a PowerShell TCP Event 3	**Unresolved**, still to test
Cause of the 2 h 11 min archive delay	**Hypothesis only** (one delayed batch, see Section 10.5); not confirmed

Lessons

- Event collection and alert generation are separate stages.
- An archived event does not prove that a rule matched, and a silent rule is not always a broken rule: check the rule's own conditions first.
- MITRE mappings must be supported by evidence.
- Normal network behaviour should not automatically be classified as malicious.
- An unresolved detection gap is a valid portfolio finding when it is documented honestly.

Next: Phase 8, Advanced Detection Engineering: custom rule matching, validation and alert generation.

10. Phase 8: Advanced Detection Engineering

Phase 7 showed that a silent rule is not always a broken rule. Phase 8 turns that lesson into a method. The question that drives it is the one a detection engineer must be able to answer for every rule: **why does this rule fire, when will it stay silent, and what harmless activity could trigger it?** This phase reviews the two custom detections confirmed as working (100101 and 100102), the built-in rules they depend on, the live rule file, and the one detection that is still unresolved (100103). Evidence was collected on 10 and 11 October 2026.

10.1 Phase 8 at a glance

Rule	Detects	Status	Evidence in this document
100101 (level 10)	`cmd.exe` starting `powershell.exe`	**Confirmed working by the author**	Rule file (Figures 5.1, 10.4), trigger event (Figure 10.3), Phase 4 process chain.
100102 (level 10, T1053.005)	`schtasks.exe` with `/create`	**Confirmed working by the author**	Alert details (Figure 5.7), alert row (Figure 11.2), rule file (Figure 10.4)
100103 (level 8)	PowerShell TCP network connection (Event 3)	**Unresolved**	Rule file (Figures 8.8, 10.4); no matching alert observed

> **Key takeaway:** this phase found that the live rule file differs from what the earlier phases documented (Section 10.8). The differences are recorded openly, and the live file is now the reference.

10.2 Objectives

1. Follow an event from collection to alert and name every stage.
2. Review the built-in parent rules that the custom rules depend on.
3. Explain the logic, benign triggers and blind spots of rules 100101 and 100102.
4. Check the live rule file against the documentation.
5. Use precise verdict terminology and an evidence-based workflow.
6. Record rule 100103 and the archive setting honestly as unresolved.

10.3 The event-to-alert pipeline



Figure 10.1: The seven stages between an endpoint event and an analyst verdict. A detection can fail at any stage, so a missing alert has to be located before it can be fixed.

Stage	What happens
1. Event generation	Windows and Sysmon record the activity, for example a process creation

Stage	What happens
2. Collection	The Wazuh agent reads the configured event channel and forwards it to the manager
3. Decoding	The manager structures the event and exposes fields such as `win.eventdata.image`
4. Parent-rule match	A built-in rule identifies the event type (Event 1, Event 3)
5. Custom-rule match	A local rule adds conditions on the image, parent or command line
6. Alert generation	When all conditions match, Wazuh writes an alert with a rule ID, level, description and event context
7. Analyst investigation	A person decides whether the behaviour is authorised, benign, suspicious or needs more work

A rule match is a detection signal. It is not proof that an attack occurred.

10.4 Rule hierarchy

The built-in Sysmon rules live in `/var/ossec/ruleset/rules/0595-win-sysmon_rules.xml`.

```
60004   Windows event channel (parent)
 └─ 61600   Sysmon base rule (severity INFORMATION)
      ├─ 61603   Event 1, process creation
      │    ├─ 100101   PowerShell from cmd.exe  [if_sid]
      │    └─ 100102   schtasks /create         [if_sid]
      └─ 61605   Event 3, network connection (level 0)
           └─ 100103   PowerShell TCP connection  [if_group]
```

Both working rules inherit from **61603** with `<if_sid>`, so their input is always a process-creation event that has already matched the parent rule. Rule 100103 is built differently: it uses `<if_group>sysmon_event3</if_group>` instead of `<if_sid>61605</if_sid>`. This structural difference is an observation, not a proven cause of the missing alert (see Section 10.12).

10.5 What the decoded fields really look like

Rules can only test fields that the decoder actually produces. Figure 10.2 shows an archived Event 1 as stored by the manager. The first Event 1 containing `parentImage` happened to be a boot-time `csrss.exe` record, so it is used here only to show field names, not as a test event.



Figure 10.2: Archived Sysmon Event 1 in archives.json, decoded by windows_eventchannel. Under data.win.eventdata the fields are written in camelCase (image, commandLine, parentImage, user, processGuid). The computer name is blacked out for publication.

- The decoder is `windows_eventchannel` and the Sysmon event ID is 1.
- Field names are camelCase: `parentImage`, `commandLine`, `processGuid`. This matters in Section 10.8.
- **Timing.** This record carries the archive timestamp `2026-10-10T06:05:48` for an event created at `03:53:14` UTC. The Opera GX Event 3 from Phase 7 was archived at `06:05:56` for an event created at `03:54:21`. Both records arrived within eight seconds of each other, about 2 hours 12 minutes after the Windows boot (the `csrss.exe` start). This suggests one backlog of events delivered together, for example after a period when the agent could not reach the manager. **That is a supported hypothesis, not a confirmed cause.**

10.6 Rule 100101: PowerShell spawned by cmd.exe

Logic. The event must match parent rule 61603 (process creation), the parent image must match `cmd.exe` and the child image must match `powershell.exe`, both case-insensitively. A match raises a **level 10** alert.

Trigger used in the lab. A fresh test was run on 11 October 2026 from a Command Prompt:

```
powershell.exe -NoProfile -Command "Get-Date"
```

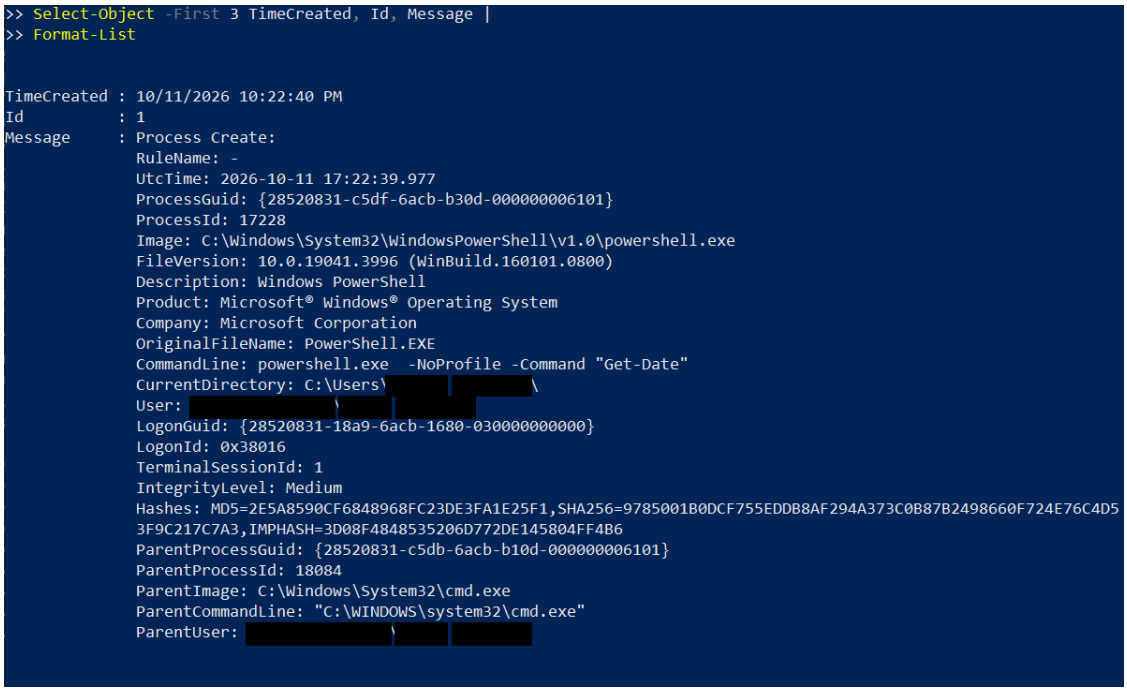


Figure 10.3: Sysmon Event 1 in Event Viewer for the test: powershell.exe with command line -NoProfile -Command "Get-Date", parent cmd.exe (PID 18084), Medium integrity. User and folder names are blacked out for publication.

Field	Value
Time (UTC)	2026-10-11 17:22:39.977 (10:22:40 PM local, UTC+5)
Image	`C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe`, PID 17228
ProcessGuid	`{28520831-c5df-6acb-b30d-000000006101}`
Parent	`C:\Windows\System32\cmd.exe`, PID 18084, ParentProcessGuid `{28520831-c5db-6acb-b10d-000000006101}`
Integrity level	Medium

This is exactly the relationship the rule describes. The command only prints the date, so the activity is a **benign true positive**: the rule is right about the process relationship, and the relationship is legitimate here.

Legitimate triggers (benign activity): an administrator opening PowerShell from Command Prompt, a batch file calling PowerShell for maintenance, installers and IT-management scripts.

Analyst checklist: read the full command line and any encoded arguments; confirm the user and whether the activity was expected; verify parent and child paths; review the process tree and nearby process-creation events; correlate file, registry, authentication and network activity by ProcessGuid and timestamp.

Limitations. The rule detects one parent-child relationship and does not prove intent. It does not match PowerShell started by a different parent or another method, and it does not inspect what runs inside PowerShell.

10.7 Rule 100102: scheduled-task creation (T1053.005)

Logic. Parent rule 61603, an image ending in `schtasks.exe`, and a command line containing `/create` as an option. A match raises a **level 10** alert mapped to MITRE ATT&CK **T1053.005, Scheduled Task/Job: Scheduled Task**. The mapping names the technique; it does not mean every detected task is malicious.

Evidence. The rule firing on a fresh test is shown in Phase 3 (Figure 5.7) and as a Wazuh Discover row in Phase 9 (Figure 11.2); the troubleshooting history is in Section 5.4.

Legitimate triggers: installers registering update tasks, administrators scheduling maintenance, security products registering scans, applications creating background tasks.

Analyst checklist: identify the task name and creating account; inspect what the task will run and where that file is; review the trigger, schedule and run-as account; look for related file creation, downloads or unusual PowerShell activity; confirm the task was authorised.

Limitations. The rule matches `schtasks.exe /create` only. Tasks can be created by other mechanisms that it does not see, and a task name alone does not show whether a task is legitimate.

10.8 Live rule file check and reconciliation

Figures 10.4 and 10.5 show `/var/ossec/etc/rules/local_rules.xml` as it was on 11 October 2026.

```
<!-- Local rules -->

<!-- Modify it at your will. -->
<!-- Copyright (C) 2015, Wazuh Inc. -->

<!-- Example -->
<group name="local,syslog,sshd,">

  <!--
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2
  -->
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>
  <rule id="100101" level="10">
    <if_sid>61603</if_sid>
    <field name="win.eventdata.parentImage" type="pcre2">(?i)cmd.exe</field>
    <field name="win.eventdata.image" type="pcre2">(?i)powershell.exe</field>
    <description>PowerShell spawned by cmd.exe</description>
    <group>sysmon,powershell,process_creation,</group>
  </rule>
  <rule id="100102" level="10">
    <if_sid>61603</if_sid>
    <field name="win.eventdata.image" type="pcre2">(?i)\\schtasks\.exe$</field>
    <field name="win.eventdata.commandline" type="pcre2">(?i)\s/create(\s|$)</field>
    <description>Scheduled task created using schtasks.exe</description>
    <mitre>
        <id>T1053.005</id>
    </mitre>
    <group>sysmon,persistence,scheduled_task.</group>
  </rule>
  <rule id="100103" level="8">
      <if_group>sysmon_event3</if_group>
      <field name="win.eventdata.image" type="pcre2">(?i)\\powershell\.exe$</field>
      <field name="win.eventdata.protocol">^tcp$</field>
      <description>powershell initiated a TCP network connection to $(win.eventdata.destinationIp):$(win.eventdata.destinationPort)</description>
      <group>sysmon_event3,powershell,network_connection,</group>
      </rule>
</group>
```

Figure 10.4: The live local_rules.xml: sample rule 100001, then custom rules 100101, 100102 and 100103, all inside one outer group.

```
7:<group name="local,syslog,sshd,">
12:  <rule id="100001" level="5">
16:    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
17:  </rule>
18:  <rule id="100101" level="10">
23:    <group>sysmon,powershell,process_creation,</group>
24:  </rule>
25:  <rule id="100102" level="10">
33:    <group>sysmon,persistence,scheduled_task.</group>
34:  </rule>
35:  <rule id="100103" level="8">
40:      <group>sysmon_event3,powershell,network_connection,</group>
41:      </rule>
42:</group>
```

Figure 10.5: Structure of the same file by line number: outer group at line 7 (local,syslog,sshd,), rules at lines 12 to 17 (100001), 18 to 24 (100101), 25 to 34 (100102), 35 to 41 (100103), group closed at line 42.

Comparing the live file with what Phases 3 and 6 and the earlier rules reference said:

Item	Earlier documentation	Live file (11 Oct)	Impact and action
100101 parent and image patterns	`(?i)\\cmd\.exe$` and `(?i)\\powershell\.exe$`	`(?i)cmd.exe` and `(?i)powershell.exe`	The live patterns are **looser**: no end anchor and the dot is unescaped, so any path or name containing `cmd` or `powershell` followed by any character and `exe` matches. The rule works for the test but could over-match. A tighter version is proposed in Section 14
100102 command-line field	Phase 3 states the field was corrected to `win.eventdata.commandLine`	`win.eventdata.commandline` (all lowercase)	**Unresolved discrepancy.** The decoder writes `commandLine` (Figure 10.2), yet the author confirms the alert fires. Either the lowercase spelling also matches, which would make the Phase 3 explanation incomplete, or the file changed after the confirmed alert. A controlled retest settles it (Section 10.12). Note that the 100102 alert in Figure 11.2 was raised on 7 October; if the file was not edited between 7 and 11 October, the lowercase spelling matched, but the screenshots cannot show that

Item	Earlier documentation	Live file (11 Oct)	Impact and action
100102 group tag	not documented	`sysmon,persistence, scheduled_task.` with a **period**	Groups are comma separated. The period creates a tag named `scheduled_task.`. Cosmetic, but it should read `scheduled_task,`
Outer group name	Section 8.7 stated `windows,sysmon,custom_network_detection,`	`local,syslog,sshd,`	Corrected in Section 8.7. This is Wazuh's sample group name. It adds those group labels to the rules and is not believed to affect matching, but it is misleading and should be renamed
Sample rule 100001	not documented	still present (`sshd` failure from 1.1.1.1)	Harmless Wazuh example; remove it or leave it, but do not confuse it with lab detections
100103 layout	identical rule	indentation differs from the other rules	Cosmetic only

The screenshot proves the state of the file on 11 October. It does not prove which version was active when each earlier alert was generated.

> **Key takeaway:** documentation must follow the live configuration. Where the two disagree, the disagreement is recorded as a finding instead of being quietly rewritten.

10.9 Verdict terminology

Term	Meaning	Lab example
Benign true positive	The rule correctly detects the configured behaviour and the behaviour is legitimate	100101 on `powershell -NoProfile -Command "Get-Date"` from `cmd.exe`
False positive	The alert indicates a condition that did not occur, because of a detection or parsing mismatch	A looser pattern matching an unrelated file name (possible with the unanchored 100101 patterns, not observed)
Suspicious	Evidence raises concern but does not establish malice	Encoded PowerShell with an unexplained scheduled task
Needs investigation	Not enough context for a reliable decision	A new task created by an unknown account
Confirmed malicious	Evidence supports malicious activity; never based on a rule match alone	None in this lab

A level 10 is the severity the rule author configured. It is not a confidence score and not proof of an attack.

10.10 Evidence-based investigation workflow

1. Read the rule description, ID, level, timestamp and agent.
2. Read the full command line and executable path.
3. Identify the user, parent process and child process.
4. Decide whether the activity is expected for that user and host.
5. Correlate nearby events by ProcessGuid and PID, using timestamps as supporting evidence.
6. Look for supporting evidence: unexpected files, persistence changes, network connections.
7. Assign a cautious verdict and write down the evidence for it.
8. Escalate or keep investigating when context is missing.

A single event, IP address, process name or MITRE mapping is never sufficient proof of malicious activity.

10.11 Detection quality review

Rule	What it covers	Known blind spots	Likely noise	Proposed improvement
100101	`cmd.exe` to `powershell.exe`	PowerShell started by other parents, other PowerShell hosts and renamed copies of the binary	Admin scripts, installers, logon scripts	Anchor the patterns; add a higher-severity variant when the command line contains `-EncodedCommand` or a download
100102	`schtasks.exe /create`	Tasks created through other interfaces or APIs, and `/change` on existing tasks	Updaters, IT tools	Add the task action (`/TR`) to the alert; raise severity when the action runs PowerShell or an unusual path
100103	PowerShell and TCP connections	Not yet working	Would be high for any PowerShell web request	Fix first; then restrict by destination or port

These are design observations from reading the rules. None of the proposed improvements has been deployed or tested yet.

10.12 Unresolved items

Rule 100103. What is established: Event 3 records are collected, archived and decoded (Section 9.2); the PowerShell TCP Event 3 of 9 October exists in the archive (Section 8.5); the rule passes `wazuh-analysisd -t`. What is **not** established: that the rule matches any event or produces an alert. The Opera GX UDP event is not a test of this rule. Untested hypotheses, in the order they would be tried:

1. Replace `<if_group>sysmon_event3</if_group>` with `<if_sid>61605</if_sid>`, matching how the working rules are built.

2. Check the `protocol` field name and value in a full archived PowerShell Event 3 against the rule condition.
3. Replay the exact archived JSON through `wazuh-logtest` (Phase 6 logtest used input the decoder did not recognise).
4. Compare with the upstream Wazuh issue #38097, which is an unconfirmed lead for this lab.

Retest of the 100102 field spelling. Change one spelling at a time, run `sudo /var/ossec/bin/wazuh-analysisd -t`, restart the manager, create a task with a new name, and record whether the alert fires. Do this on a copy of the file and note the result for both spellings.

Archive setting. `<logall_json>` was temporarily set to `yes` for troubleshooting. Its current value has not been verified. Check it with `grep -n "logall" /var/ossec/etc/ossec.conf`, restore the intended value, validate the configuration and restart the manager. Archives did contain data on 10 October (Figure 10.2), which says nothing about today's setting.

10.13 Phase 8 status

Item	Status
Pipeline and rule hierarchy documented	Complete
Rules 100101 and 100102 explained, with benign triggers and blind spots	Complete
Rule 100101 trigger event captured	Complete (Figure 10.3)
Live rule file captured and compared with the documentation	Complete, discrepancies recorded
100102 field spelling retest	Planned
Rule 100103	**Unresolved**
`logall_json` current value	**Not verified**

Lessons

- Document the live configuration, not the configuration you remember.
- A rule that works is not necessarily a well-written rule: looser patterns, odd tags and unexplained spelling differences are technical debt.
- Use precise verdict terms and never treat severity as confidence.
- An unresolved detection is a valid portfolio finding when it is recorded honestly.

Portfolio principle: document real observations, label unresolved work clearly, and base verdicts on correlated evidence rather than alert severity.

11. Phase 9: Detection Validation and SOC Investigation

Phases 7 and 8 asked whether a detection works. Phase 9 asks the analyst's question that comes after an alert: **what does the evidence actually prove?** The case revisits the scheduled task from Phase 3 (`Wazuh-Day03-Test-4`) five days later. It starts from a real Wazuh alert, follows it through the Sysmon record and the live task configuration, and ends with a verdict that separates what was **observed** from what was only **inferred**. The investigation was carried out on 12 October 2026.

11.1 Case at a glance

Item	Detail
Case	Review of a scheduled task created by `schtasks.exe /Create`
Host / agent	`WIN10-SOC-LAB` (ID 001), Windows 10 Pro
Detection	Custom rule **100102**, level 10, MITRE ATT&CK T1053.005
Task under review	`\Wazuh-Day03-Test-4`, action `cmd.exe /c echo DAY03-PERSISTENCE-TEST-4`, trigger at logon
Verdict	**Likely benign lab activity; execution details not fully corroborated**
Evidence limitation	No Sysmon record of the task action itself was found in the evidence reviewed

> **Key takeaway:** an alert shows that something matched a rule. It does not show what happened afterwards. No alert for an action does not mean the action did not happen, and it does not mean Wazuh failed.

11.2 Events, alerts and execution evidence

A Windows event can be collected by Wazuh and be available for investigation without producing an alert. An alert exists only when an event passes through analysis and matches a rule that raises one. Three different things are easy to confuse in this case:

Artefact	What it is	What it proves
Sysmon Event 1 for `schtasks.exe /Create`	A process-creation event	The command that created the task was run
Wazuh alert for rule 100102	A detection raised from that event	The event matched the rule's conditions
Event for the task's action (`cmd.exe /c echo ...`)	A separate process-creation event, only if the task ran	That the action actually executed. **Not found in this case**

An alert is not created automatically for every process or every scheduled-task run. It depends on the collected event and on the rules that exist.

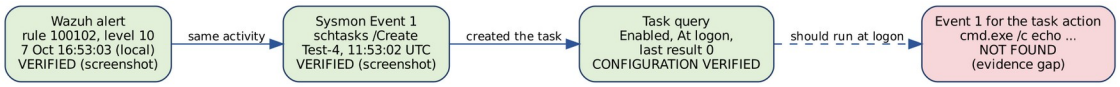


Figure 11.1: The evidence chain for the case. Three links are verified; the last link, execution of the task action, was not corroborated.

11.3 Evidence A: the alert and the creation event

The Wazuh alert. Rule 100102 fired for the creation of the task on 7 October 2026.

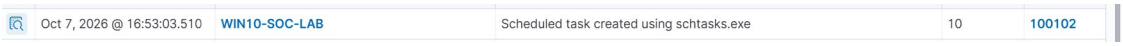


Figure 11.2: Wazuh Discover row for rule 100102 on agent WIN10-SOC-LAB: "Scheduled task created using schtasks.exe", level 10, 7 Oct 2026 16:53:03.510 (local time, UTC+5).

The Sysmon record behind it. The matching Event 1 was inspected in Wazuh Discover (field `data.win.system.message`):

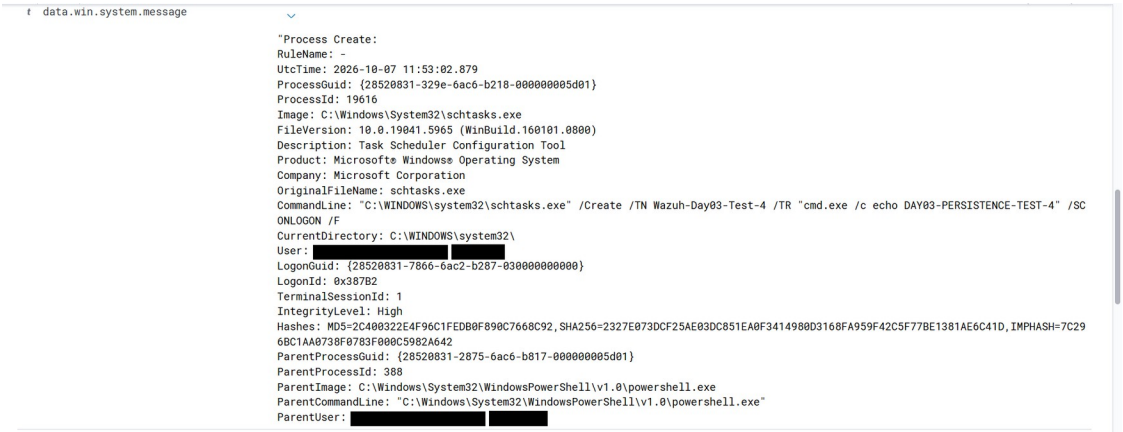


Figure 11.3: Sysmon Event 1 for schtasks.exe /Create /TN Wazuh-Day03-Test-4 ... /SC ONLOGON /F, parent PowerShell, High integrity. User names are blacked out for publication.

Field	Value
Event ID	1 (process create)
Time	2026-10-07 11:53:02.879 UTC
Image / PID	`C:\Windows\System32\schtasks.exe`, 19616
ProcessGuid	`{28520831-329e-6ac6-b218-000000005d01}`
Parent	`C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe`, PID 388
Integrity level	High

Command line:

```
"C:\WINDOWS\system32\schtasks.exe" /Create /TN Wazuh-Day03-Test-4 /TR "cmd.exe /c echo DAY03-PERSISTENCE-TEST-4" /SC ONLOGON /F
```

Reading the two together. The alert (16:53:03.510 local) is 0.63 seconds after the process creation (11:53:02.879 UTC is 16:53:02.879 local). The alert row does not show the ProcessGuid, so the link rests on

time, agent, rule and task name, which is supporting evidence rather than a proof by identifier. The process-creation record shows `/Create` with the stated action, a PowerShell parent and high integrity. This is the same test run from PowerShell that Phase 3 recorded (Figure 5.7); the earlier first test in Phase 3 used `cmd.exe` as the parent.

What this proves: Sysmon recorded the command that created the task, and rule 100102 raised its alert.
What it does not prove: that the task's action ever executed.

11.4 Evidence B: the task as it exists now

The task was inspected with a read-only query:

```
schtasks /Query /TN "Wazuh-Day03-Test-4" /V /FO LIST
```

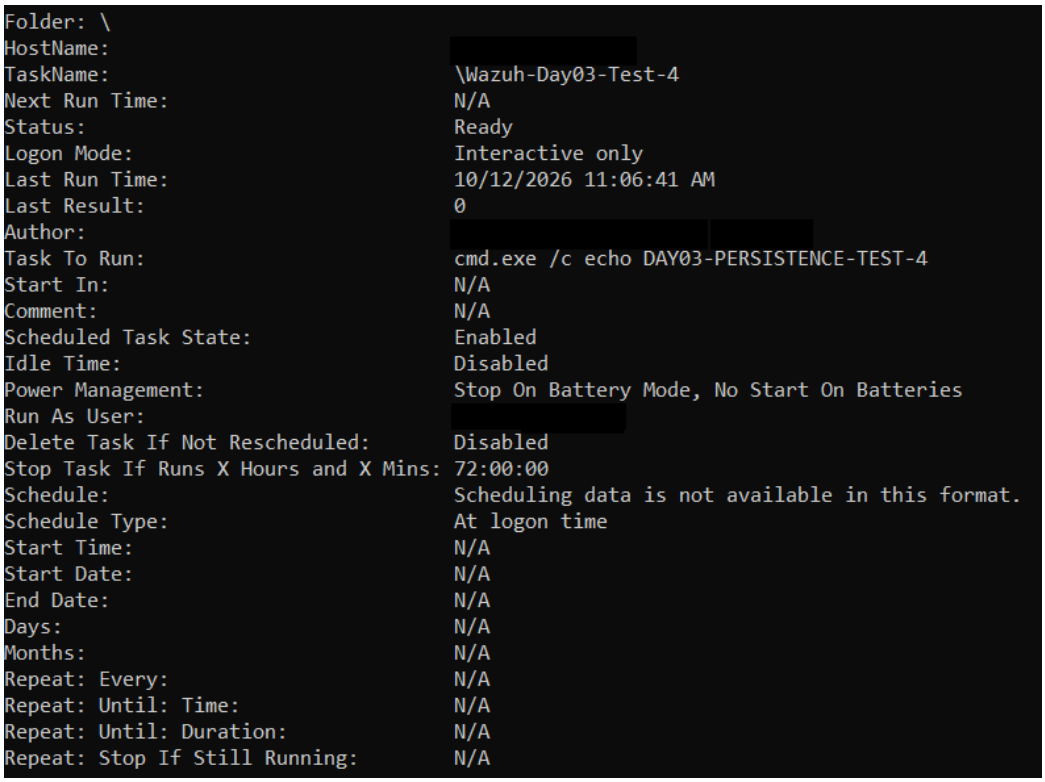


Figure 11.4: Verbose task query for -Day03-Test-4: Ready, Enabled, trigger At logon time, action cmd.exe /c echo DAY03-PERSISTENCE-TEST-4, last run 10/12/2026 11:06:41 AM, last result 0. Host and user names are blacked out for publication.

Field	Value
Status / state	Ready / Enabled
Schedule type	At logon time
Task to run	`cmd.exe /c echo DAY03-PERSISTENCE-TEST-4`
Last run time	10/12/2026 11:06:41 AM (local), which is 06:06:41 UTC
Last result	`0`
Run mode	Interactive only, power settings: stop on battery mode

The configuration matches the intended test, and a last result of 0 is consistent with a successful run. But the query is a snapshot of the task's settings. It does not show which process ran, under which parent, or what its command line was. It is useful evidence, not a corroborated execution.

11.5 Evidence C: the query is also an event

Running the query above created its own Sysmon Event 1.

Field	Value
Time	2026-10-12 06:36:57.756 UTC (11:36:57 AM local)
Image / PID	`C:\Windows\System32\schtasks.exe`, 10672
Parent	`C:\Windows\System32\cmd.exe`
Integrity level	Medium
Command line	`schtasks /Query /TN "Wazuh-Day03-Test-4" /V /FO LIST`

The command line contains `/Query`, not `/Create`, and it does not contain the echo action. This record is evidence that the task was **queried**, not that its action ran. It also gives a useful negative check: rule 100102 requires `/create`, so this event should not alert, and no alert for it is expected.

11.6 Timeline

UTC	Local (UTC+5)	Observation	Reading
2026-10-07 11:53:02	7 Oct 16:53:02	Sysmon Event 1: `schtasks.exe /Create`	The task-creation process was observed
2026-10-07 11:53:03	7 Oct 16:53:03	Wazuh alert, rule 100102	The creation event matched the rule
2026-10-12 06:06:41	12 Oct 11:06:41	Task query reports this as the last run time	Windows reports a run at logon; its process was **not** independently corroborated
2026-10-12 06:36:57	12 Oct 11:36:57	Sysmon Event 1: `schtasks.exe /Query`	The task was queried. This is not the task action

The UTC values for the 12 October rows are the local times converted by 5 hours. These are separate observations and must not be merged into one execution.

11.7 Does the missing alert mean Wazuh failed?

Not necessarily. Reasons that follow from how the lab is built:

1. Rule 100102 matches `schtasks.exe` with `/create`. Running the task later does not create another `/create` event.

2. The task's action is an ordinary `cmd.exe` process creation. No custom rule in this lab matches it, so it would be an archived event at most, not an alert (the archive/alert distinction from Phases 7 and 8).
3. The matching process-creation event was never located, so the case cannot show that Wazuh missed an event it should have alerted on.

The correct response is to record the missing evidence, not to assume the task failed or that a rule is broken. **No rule was changed to force an alert for this case.**

11.8 Verdict

> **Verdict:** Likely benign lab activity; execution details not fully corroborated.

Evidence supporting the verdict: the task name matches the lab exercise; the action is a harmless echo command; the configuration matches the intended test; Windows reports last result 0; the investigation was part of the planned lab.

Limitations:

- The process-creation event for the configured action was not confirmed.
- The last-run time does not establish the process tree of that run.
- No separate Wazuh execution alert was seen.
- The conclusion covers this case and this evidence only. It does not say the whole host was free of suspicious activity.

This is a preliminary SOC verdict, not a claim that every detail has been proven.

11.9 Closing the evidence gap

Steps that would produce stronger evidence. They are **planned and not yet done**.

1. Preserve the rule 100102 alert, the original Sysmon creation event and the query output (done for this document: Figures 11.2 to 11.4).

2. Search the local Sysmon log around the reported last run for the action's command line (times are local):

```
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational';
Id=1;
  StartTime='2026-10-12 11:00'; EndTime='2026-10-12 11:15'} |
  Where-Object { $_.Message -like '*DAY03-PERSISTENCE-TEST-4*' } |
  Format-List TimeCreated, Message
```

3. Search Wazuh for the same string in the same window. A cmd.exe record would appear only in the archive, and only if archiving was enabled then; the `logall_json` setting is still unverified (Section 10.12).

4. Open Task Scheduler history (Event Viewer, Microsoft, Windows, TaskScheduler, Operational) around 11:06:41. History must have been enabled before the run to contain it.

5. If no matching event exists, record that as an evidence gap.

6. Once the evidence is preserved and the task is confirmed as the lab task, remove it so it does not persist: `schtasks /Delete /TN "Wazuh-Day03-Test-4" /F`. The other Phase 3 test tasks need the same review.

11.10 Phase 9 status and lessons

Item	Status
Difference between collected events and alerts	Demonstrated
Alert, creation event and task configuration reviewed	Complete (Figures 11.2 to 11.4)
Query event identified correctly as `/Query`	Complete
Independent corroboration of the task action	**Open**, documented as an evidence gap

Lessons

1. An alert is not proof of an attack, and a process-creation event is not proof of malicious activity.
2. A task's reported last result is useful, but it is not independent corroboration of process execution.
3. A missing alert does not prove that something did not happen.
4. Your own investigation commands create events; label them so they are not mistaken for the activity under review.
5. Record what is known, what is inferred and what is unverified.

Portfolio principle: a good SOC analyst does not invent certainty. The quality of an investigation is accurate evidence handling, correlation and a conclusion that can be defended.

12. Phase 10: File Integrity Monitoring (FIM) Investigation

Process, network and authentication telemetry show what *runs* and who *logs on*. File Integrity Monitoring answers a different question: **what changed on disk, and can the change be proven?** Wazuh's FIM module (syscheck) records a baseline of a file's size, timestamps, permissions and cryptographic hashes, and raises an alert when a monitored file is added, modified or deleted. This phase checks the agent's FIM configuration, runs a controlled create, modify and delete test, cross-checks every alert against independent evidence, and records one older alert that could not be judged. The work was done on 12 October 2026. Times in this section are **local time, UTC+5**, as displayed by the Wazuh dashboard.

12.1 Phase 10 at a glance

Question	Evidence	Answer
Is FIM enabled on the Windows agent?	`ossec.conf` (Figures 12.2, 12.3)	**Yes**; scheduled scan every 12 hours, Startup folder in real time
Does Wazuh detect a new file?	Rule 554 alert, level 5 (Figures 12.5 to 12.8)	**Yes**; size and SHA-256 recorded
Do the recorded hashes match reality?	PowerShell `Get-FileHash` (Figure 12.9)	**Yes**; SHA-256 identical
Does Wazuh detect a modification?	Level 7 alert, `Changed attributes: size, mtime, md5, sha1, sha256` (Figures 12.11, 12.12)	**Yes**; before and after values recorded
Does Wazuh detect a deletion?	Level 7 "File deleted." alert (Figures 12.14, 12.15)	**Yes**; last known state preserved
Was an older FIM alert on the SYSTEM account explained?	Details without path or change type	**No: inconclusive**

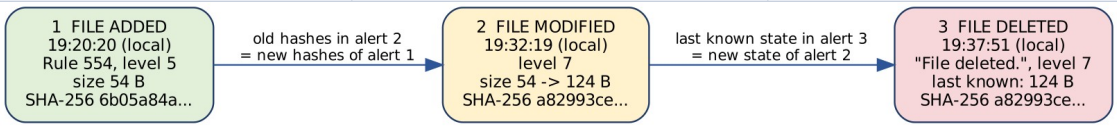


Figure 12.1: The test file's lifecycle as seen by Wazuh. Each alert carries the previous state, so the three alerts form a chain.

> **Key takeaway:** FIM proves *that* a file changed and *what* the change was, down to the hash. It does not by itself say *which process* changed it, so a FIM alert is a starting point that must be correlated with process evidence.

12.2 Why FIM matters to a SOC analyst

- **Tampering:** modified system binaries (for example `cmd.exe`, `powershell.exe`, `schtasks.exe`, the same tools abused in Phases 3 and 4) can indicate an attacker or a corrupted system.
- **Persistence:** files placed in a Startup folder can be configured to run at sign-in (MITRE ATT&CK T1547.001, Boot or Logon Autostart Execution: Registry Run Keys / Startup Folder). A file's location alone does not show that it runs or that it is malicious.

- **Evidence integrity:** hashes let an analyst show that a file is exactly the one that was recorded, and detect a silent change.
- **Compliance:** the alerts carry PCI DSS, NIST 800-53, GDPR, HIPAA and TSC tags (visible in Figure 12.7).

12.3 FIM configuration review

The Windows agent's local configuration was inspected with `Select-String` on `C:\Program Files (x86)\ossec-agent\ossec.conf`.

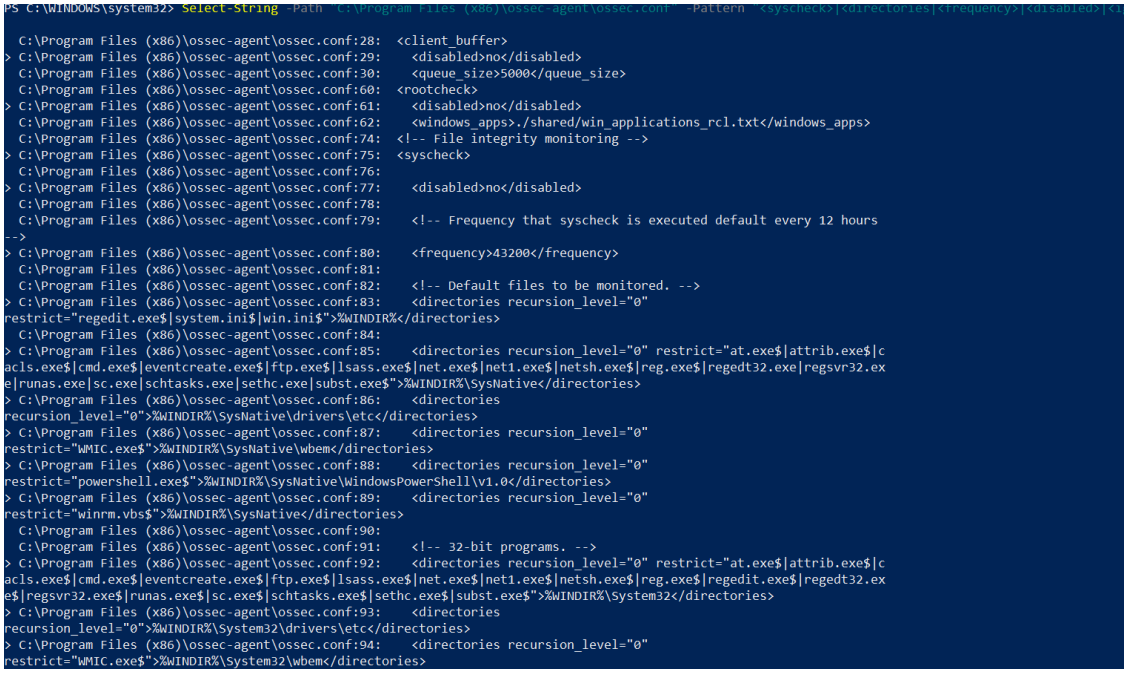


Figure 12.2: Start of the syscheck block: the section opens at line 75 with FIM enabled (`<disabled>no</disabled>`, line 77) and a scan frequency of 43200 seconds, 12 hours (line 80). Lines 83 to 94 list the monitored Windows directories.

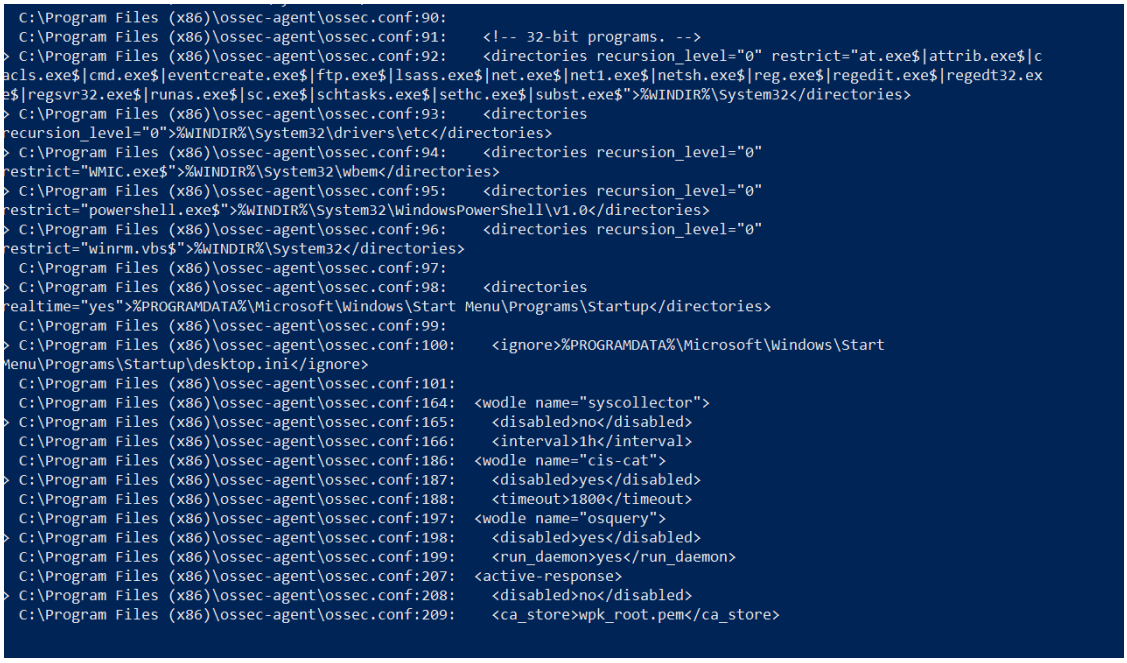


Figure 12.3: Rest of the monitored paths. Line 98 monitors the user Startup folder with realtime="yes"; line 100 ignores `desktop.ini` there. Lines 164 onward show other modules (syscollector enabled, CIS-CAT and osquery disabled, active response enabled).

Setting	Observed value	Meaning
FIM module	`<disabled>no</disabled>`	Enabled
Scheduled scan	`<frequency>43200</frequency>`	A full scan every 12 hours
Windows system directories	`%WINDIR%`, `System32`, `SysNative`, `drivers\etc`, `wbem`, `WindowsPowerShell\v1.0`	Watched, with a `restrict` list of sensitive file names (for example `cmd.exe`, `schtasks.exe`, `powershell.exe`, `reg.exe`, `net.exe`)
Startup folder	`%PROGRAMDATA%\Microsoft\Windows\Start Menu\Programs\Startup`, `realtime="yes"`	Watched **in real time**
Excluded file	`...\Startup\desktop.ini`	Ignored to reduce noise

What the configuration implies.

- Only the Startup folder is real time. A change to a system binary such as `schtasks.exe` is found at the **next scheduled scan**, which can be up to 12 hours later. That is a detection-speed trade-off an analyst must know about.
- The restricted file names are exactly the Windows tools used in earlier phases. FIM watches whether those programs are *altered*, while Sysmon (Phases 3 and 4) watches when they are *run*. The two views complement each other.

- The same file also shows `<client_buffer>` enabled with a queue of 5000 events (lines 29 and 30). A buffer holds events when the agent cannot reach the manager. This is relevant background to the delayed archive records in Sections 9.2 and 10.5, but it is **not** evidence of their cause.

The Startup folder was therefore the natural place for a controlled test: it is monitored in real time and relevant to persistence.

12.4 Test design

A harmless text file was created in the monitored folder, modified once, then deleted:

```
C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup\wazuh-fim-test.txt
```

Step	Action	Time (local)	Wazuh alert
1	Create the file (54 bytes)	19:20:19	“File added to the system.”, rule 554, level 5, at 19:20:20.070
2	Verify the SHA-256 with PowerShell	shortly after	n/a (independent check)
3	Append one line (file grows to 124 bytes)	19:32:19	Modified, level 7, at 19:32:19.558
4	Delete the file	not timestamped in the screenshot	“File deleted.”, level 7, at 19:37:51.941

The file is a `.txt` and does not execute. A real Startup-folder persistence attempt would place an executable, script or shortcut there; this test only exercises the detection path.

12.5 Test 1: file creation

```
PS C:\WINDOWS\system32> $testFile = Join-Path $startup "Wazuh-FIM-Test.txt"
>> "Initial FIM test - $(Get-Date -Format o)" | Set-Content -Path $testFile
>> Get-Item $testFile | Select-Object FullName, Length, LastWriteTime
>> Get-FileHash $testFile -Algorithm SHA256

FullName                                                                Length LastWriteTime
--------                                                                ------ -------------
C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup\Wazuh-FIM-Test.txt     54 10/12/2026 7:20:19 PM
```

Figure 12.4: Creating the test file: `Set-Content` writes one line, `Get-Item` shows length 54 and LastWriteTime 7:20:19 PM, and `Get-FileHash` is called to fingerprint it.

Wazuh raised the alert about one second after the file’s write time:

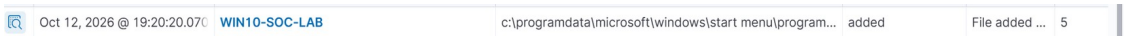


Figure 12.5: Wazuh Discover row: 12 Oct 2026 19:20:20.070, agent WIN10-SOC-LAB, the Startup path, event “added”, “File added …”, level 5.

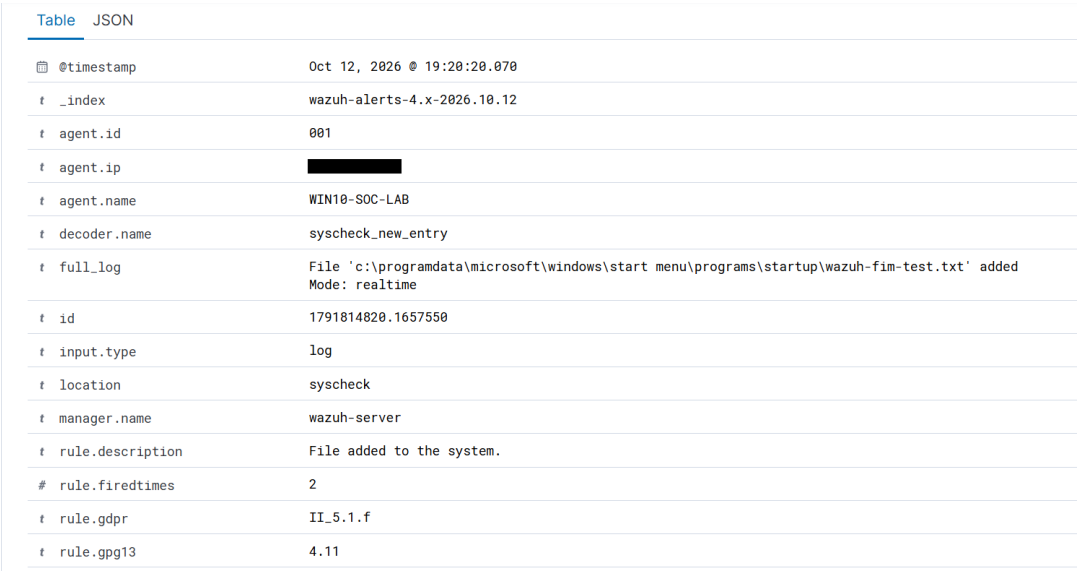


Table JSON

@timestamp	Oct 12, 2026 @ 19:20:20.070
_index	wazuh-alerts-4.x-2026.10.12
agent.id	001
agent.ip	[redacted]
agent.name	WIN10-SOC-LAB
decoder.name	syscheck_new_entry
full_log	File 'c:\programdata\microsoft\windows\start menu\programs\startup\wazuh-fim-test.txt' added Mode: realtime
id	1791814820.1657550
input.type	log
location	syscheck
manager.name	wazuh-server
rule.description	File added to the system.
rule.firedtimes	2
rule.gdpr	II_5.1.f
rule.gpg13	4.11

Figure 12.6: Alert details, upper part: index `wazuh-alerts-4.x-2026.10.12`, agent 001, decoder `syscheck_new_entry`, location `syscheck`, manager `wazuh-server`, description "File added to the system.", full log "File … added, Mode: realtime".

rule.gpg13	4.11
rule.groups	ossec, syscheck, syscheck_entry_added, syscheck_file
rule.hipaa	164.312.c.1, 164.312.c.2
rule.id	554
rule.level	5
rule.mail	false
rule.nist_800_53	SI.7
rule.pci_dss	11.5
rule.tsc	PI1.4, PI1.5, CC6.1, CC6.8, CC7.2, CC7.3
syscheck.attrs_after	ARCHIVE, NOT_CONTENT_INDEXED
syscheck.event	added
syscheck.md5_after	c2fd59d5f3701aa0eb2c7c3b8ff40970
syscheck.mode	realtime
syscheck.mtime_after	Oct 12, 2026 @ 19:20:19.000
syscheck.path	c:\programdata\microsoft\windows\start menu\programs\startup\wazuh-fim-test.txt
syscheck.sha1_after	da385fc5bc77b857f03fa7699d2ee779f4c9905f
syscheck.sha256_after	6b05a84a5253d496ce67a306743ce967b4414384ec3853acac6ba15d5d90619c

Figure 12.7: Alert details, rule fields: rule ID 554, level 5, groups `ossec, syscheck, syscheck_entry_added, syscheck_file`, compliance tags (GDPR, GPG13, HIPAA, NIST 800-53 SI.7, PCI DSS 11.5, TSC), `syscheck.event: added`, mode realtime, the file path, mtime 19:20:19, and the MD5, SHA-1 and SHA-256 values.



Figure 12.8: Alert details, file attributes: size 54 bytes, owner SID S-1-5-32-544 (Administrators), the Windows permission list and the alert timestamp. User names in the permission list are blacked out for publication.

Field	Value in the alert
Rule / level	554 / 5, “File added to the system.”
Decoder / mode	`syscheck_new_entry` / realtime
Size	54 bytes
Modification time	12 Oct 2026 19:20:19
Owner	Administrators (S-1-5-32-544)
MD5	`c2fd59d5...ff40970`
SHA-1	`da385fc5...c9905f`
SHA-256	`6b05a84a5253d496ce67a306743ce967b4414384e c3853acac6ba15d5d90619c`

Hash verification. The same file was fingerprinted with PowerShell:

```
Algorithm : SHA256
Hash      : 6B05A84A5253D496CE67A306743CE967B4414384EC3853ACAC6BA15D5D90619C
Path      : C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup\wazuh-fim-test.txt
```

Figure 12.9: `Get-FileHash -Algorithm SHA256` on the test file returns 6B05A84A…5D90619C, the same value Wazuh recorded in the file-added alert (hex is case-insensitive).

The two SHA-256 values are identical, so the file’s content at verification time matched the fingerprint stored in the original event. As a size check, the first line is 19 characters of text plus a 33-character ISO timestamp plus a 2-byte line break, which is 54 bytes, the size Wazuh recorded.

> **Key takeaway:** an independent tool and the Wazuh alert agree on the same SHA-256, so the alert can be trusted as a faithful record of the file.

12.6 Test 2: file modification

```
PS C:\WINDOWS\system32> Add-Content -Path $path -Value "Controlled FIM modification test - $(Get-Date -Format o)"
>> Get-Item $path | Select-Object FullName, Length, LastWriteTime
>> Get-FileHash $path -Algorithm SHA256

FullName                                                        Length LastWriteTime
--------                                                        ------ -------------
C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup\wazuh-fim-test.txt    124 10/12/2026 7:32:19 PM
```

Figure 12.10: Appending a line with `Add-Content`, then `Get-Item` shows length 124 and LastWriteTime 7:32:19 PM.

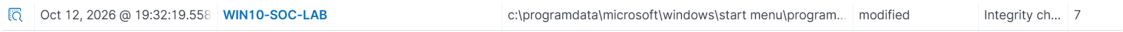


Figure 12.11: Wazuh Discover row: 12 Oct 2026 19:32:19.558, event "modified", description beginning "Integrity ch…", level 7.

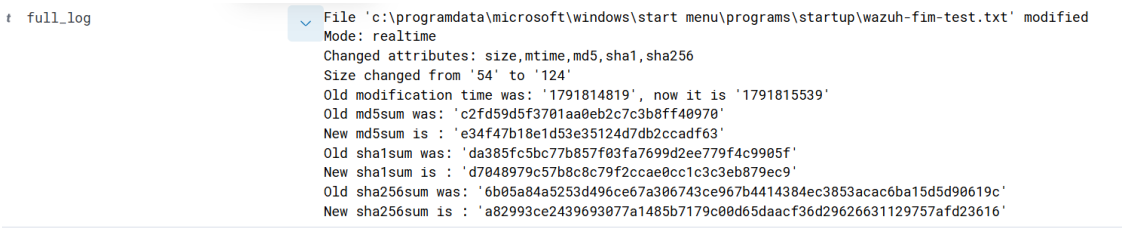


Figure 12.12: Full log of the modification alert: mode realtime; changed attributes size, mtime, md5, sha1, sha256; size changed from 54 to 124; old and new modification times; old and new MD5, SHA-1 and SHA-256.

Attribute	Before	After
Size	54 bytes	124 bytes (+70)
Modification time	1791814819 = 19:20:19 local	1791815539 = 19:32:19 local (12 minutes later)
MD5	`c2fd59d5...ff40970`	`e34f47b1...ccadf63`
SHA-1	`da385fc5...c9905f`	`d7048979...b879ec9`
SHA-256	`6b05a84a...5d90619c`	`a82993ce...afd23616`

Cross-checks that passed.

- The "before" hashes in the modification alert are exactly the "after" hashes of the file-added alert (Figure 12.7 against Figure 12.12). The baseline chain is unbroken.
- The size grew by 70 bytes, which equals the appended text (35 characters, a 33-character timestamp and a 2-byte line break). The new size matches `Get-Item` (124).
- The epoch times convert to 19:20:19 and 19:32:19 local, matching the PowerShell write times and a 12-minute gap.

What is not independently verified. The new SHA-256 (`a82993ce...`) is taken from Wazuh's alerts and was not re-computed with a second tool, so it rests on Wazuh alone. The deletion alert repeats the same value (next section), but that is Wazuh again, not a second tool.

The rule ID of the modification and deletion alerts is not visible in the screenshots (the rows show only the description and level 7). Wazuh's standard FIM rules for these events are 550 (checksum changed) and 553 (file deleted); this should be confirmed in Discover before the IDs are quoted.

12.7 Test 3: file deletion

The notes asked that deletion be recorded only after its alert had been verified. The alert was captured, so it is recorded here.

```
PS C:\WINDOWS\system32> Remove-Item -LiteralPath $path
>> Test-Path $path
False
```

Figure 12.13: Deleting the file with `Remove-Item -LiteralPath $path`; `Test-Path $path` returns False.

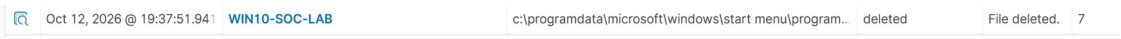


Figure 12.14: Wazuh Discover row: 12 Oct 2026 19:37:51.941, event "deleted", "File deleted.", level 7.



*Figure 12.15: Alert details for the deletion: `syscheck.event: deleted`, mode realtime, and the **last known** attributes of the file: size 124, modification time 19:32:19, MD5 `e34f47b1...`, SHA-1 `d7048979...`, SHA-256 `a82993ce...`. User names in the permission list are blacked out for publication.*

The deleted file's last known size, modification time and all three hashes equal the **after** values of the modification alert. The three alerts therefore form one consistent chain: added (54 bytes), modified (124 bytes), deleted (last state 124 bytes). `Test-Path` returning False shows the file really was gone, independent of Wazuh.

12.8 Existing alert on the SYSTEM account: inconclusive

Before the controlled tests, an older FIM alert was reviewed. Its available details were:

Detail	Value
User / SID	SYSTEM / S-1-5-18
Size	219 bytes
Last modified	19 March 2019
Hashes and permissions	MD5, SHA-1, SHA-256 and a Windows permission list
File path	**not available** in the details supplied
Change type (added, modified, deleted)	**not available**

Verdict: inconclusive. Without the path and the change type, the alert cannot be called benign or malicious. The `SYSTEM` account and the permissions do not decide it: legitimate Windows servicing runs as `SYSTEM`, and

so would an attacker with high privileges. A modification time of 2019 describes the file's content timestamp, not when the change happened.

How the case could be closed. Search Discover for the alert (user `SYSTEM`, size 219) and read `syscheck.path` and `syscheck.event`. Then check whether the path is a normal Windows component, compare the hash with a trusted reference, look at what else happened on the host at that time (Sysmon Event 11 file creation and Event 1 process creation for the same minute), and check whether a patch or installation was running. If none of this is possible, record the alert as an evidence gap.

12.9 Analysis of the controlled test

Verdict for the test: benign, controlled lab activity. All three alerts match actions the author performed, with consistent times, sizes and hashes.

What FIM showed well: the exact event (added, modified, deleted), size and timestamp changes, before-and-after hashes, and the file owner and permissions, all within about a second in real-time mode.

What FIM did not show. None of the captured fields identifies the **process or user that made the change**. In this evidence the alert shows the file's owner (Administrators), not the program that wrote it. Attribution would need one of: Wazuh's who-data monitoring (not enabled in the configuration reviewed), or correlation with Sysmon Event 11 (file creation) and Event 1 (process creation) using the time window, as practised in Phase 4. Checking Event 11 for `wazuh-fim-test.txt` around 19:20:19 is a planned follow-up, not something done in this phase.

Limits of the test. A text file does not execute. Only the Startup folder was tested in real time; system binaries are checked every 12 hours, so a change there would be reported late. A real attacker could also place a file outside the monitored directories.

12.10 Detection improvements (proposed, not deployed)

1. **Severity by file type.** A new `.exe`, `.bat`, `.cmd`, `.ps1`, `.vbs` or `.lnk` in the Startup folder is far more interesting than a text file. A custom rule built on the file-added rule (554) could raise those to a higher level. The rule below is a **sketch**: the field name and pattern must be validated against a real Windows FIM alert and with `wazuh-analysisd -t` before use.

```
<rule id="100110" level="10">
  <if_sid>554</if_sid>
  <field name="file" type="pcre2">(?i)\\start menu\\programs\\startup\\.+\.(exe|bat|
cmd|ps1|vbs|lnk)$</field>
  <description>Executable or script added to a Startup folder</description>
  <mitre><id>T1547.001</id></mitre>
  <group>syscheck,persistence,startup_folder,</group>
</rule>
```

2. **Faster coverage of sensitive binaries.** Consider real-time monitoring for the directories holding the restricted file names, accepting the extra load, or shortening the 12-hour scan interval.

3. **Attribution.** Enable who-data monitoring for the Startup folder, or build a documented correlation step with Sysmon Event 11.

4. **Noise control.** Keep an ignore list (like `desktop.ini`) for known benign Startup entries, and review it periodically.

12.11 Phase 10 status and lessons

Item	Status
FIM configuration reviewed	Complete (Figures 12.2, 12.3)
File creation detected and hash verified	Complete (Figures 12.5 to 12.9)
File modification detected	Complete; new hash not independently verified
File deletion detected	Complete (Figures 12.13 to 12.15)
Existing SYSTEM alert (219 bytes)	**Inconclusive**
Process attribution of the file changes	**Not established** (planned)

Lessons

1. A FIM alert proves a change, not intent, and not which process caused it.
2. Verify what the tool reports with an independent tool (here, `Get-FileHash`), and keep track of what was *not* independently verified.
3. Each alert carries the previous state, so a sequence of FIM alerts can be checked for consistency like a chain of custody.
4. Real-time and scheduled monitoring differ in detection delay; know which applies to the path you care about.
5. An alert without its path and change type is inconclusive. Record it as such instead of guessing.

Portfolio principle: every finding is tied to a screenshot or an arithmetic check; what could not be verified is written down as such.

13. Phase 11: Vulnerability Detection and Remediation

Earlier phases looked at what *happens* on the endpoint: processes, logons, network connections and file changes. Vulnerability detection asks a different question: **what weaknesses exist on the endpoint before anything happens?** Wazuh compares the software inventory reported by the agent with a database of known vulnerabilities (CVEs) and lists every installed package whose version is affected. This phase takes one High-severity finding from the dashboard, proves what it means, fixes it, and then checks that Wazuh agrees the problem is gone. A stale inventory record delayed that last step, so the phase also records how the discrepancy was investigated. The work was done on 13 October 2026. Times in this section are **local time, UTC+5**, unless stated.

13.1 Phase 11 at a glance

Question	Evidence	Answer
What does Wazuh report for the endpoint?	Dashboard counts (Figure 13.2)	**91 findings**: 0 Critical, 22 High, 57 Medium, 12 Low
Which software does the chosen finding affect?	Finding details (Figure 13.3)	**7-Zip 25.01 (x64)**, `CVE-2026-48111`
Is the software really installed in that version?	PowerShell registry query (Figure 13.4)	**Yes**, 25.01, in `C:\Program Files\7-Zip\`
Was it fixed?	Registry query after the update (Figure 13.5)	**Yes**, 26.04
Did Wazuh follow immediately?	Package inventory (Figure 13.6)	**No**: the inventory still showed 25.01 for a while
Was the agent healthy?	Agent panel (Figure 13.7), log and connectivity checks in the notes	**Active**; earlier connection errors found in the agent log
Is the finding closed?	Inventory and CVE search (Figures 13.8, 13.9)	**Yes**: inventory shows 26.04 and the CVE is no longer listed

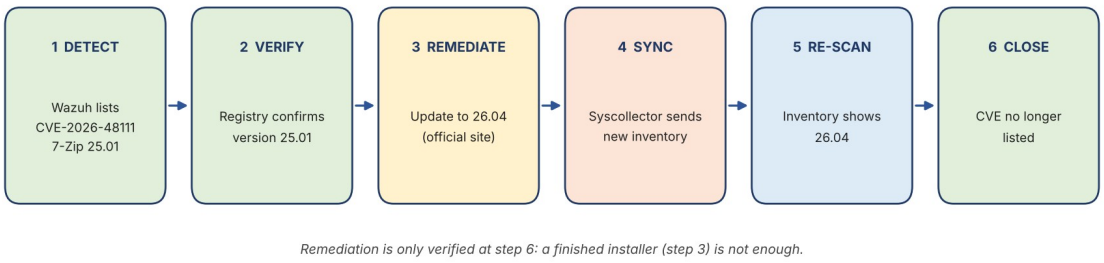


Figure 13.1: The vulnerability workflow practised in this phase. Remediation counts as verified only at the last step, when the scanner itself no longer lists the finding.

> **Key takeaway:** a vulnerability finding is a **weakness**, not an **incident**. The job is to confirm it, fix it and prove the fix, without confusing “version matches a CVE” with “the host was attacked”.

13.2 How Wazuh finds vulnerabilities, and why the inventory matters

The agent's Syscollector module collects the list of installed software (and other inventory data) and sends it to the manager. The Vulnerability Detection module then matches package names and versions against vulnerability content, in this case the National Vulnerability Database as stated in the author's notes, and shows each match with a CVE ID, a CVSS score and the fixed version.

Two practical consequences follow, and both showed up in this phase:

- **The scanner is only as current as the inventory.** If the inventory still says 25.01, the scanner keeps reporting 25.01 even when Windows already has 26.04 (Section 13.7).
- **A match is a version comparison, not a test.** Nothing was exploited and nothing was proved about the host; the scanner only found a version that falls inside the vulnerable range.

(This paragraph describes how the module works in general. What was *observed* in this lab is shown in the figures that follow.)

13.3 Step 1: the findings overview

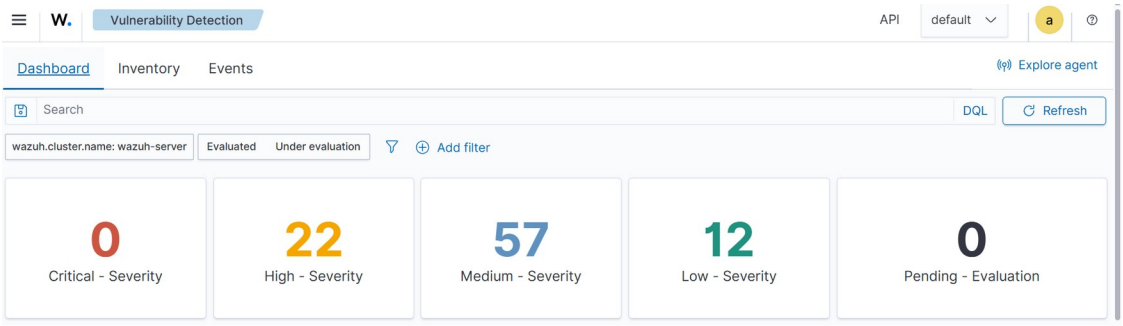


Figure 13.2: Vulnerability Detection dashboard: 0 Critical, 22 High, 57 Medium, 12 Low and 0 Pending evaluation. The filter bar shows the cluster `wazuh-server` and the Evaluated / Under evaluation switch; no agent filter is applied.

Severity	Findings
Critical	0
High	22
Medium	57
Low	12
Pending evaluation	0
Total	**91** (equal to the 91 records in the author's notes)

The dashboard in Figure 13.2 is not filtered to agent 001. The lab has had a single endpoint throughout this document, so the counts are treated as that endpoint's; adding an `agent.id: 001` filter would confirm it.

The author opened one High finding to learn what the data contains and what a fix involves. Choosing one finding and following it end to end is the realistic way to work: 91 findings cannot all be fixed in one sitting, and the first decision is *which one to take first* (Section 13.10).

13.4 Step 2: reading one finding

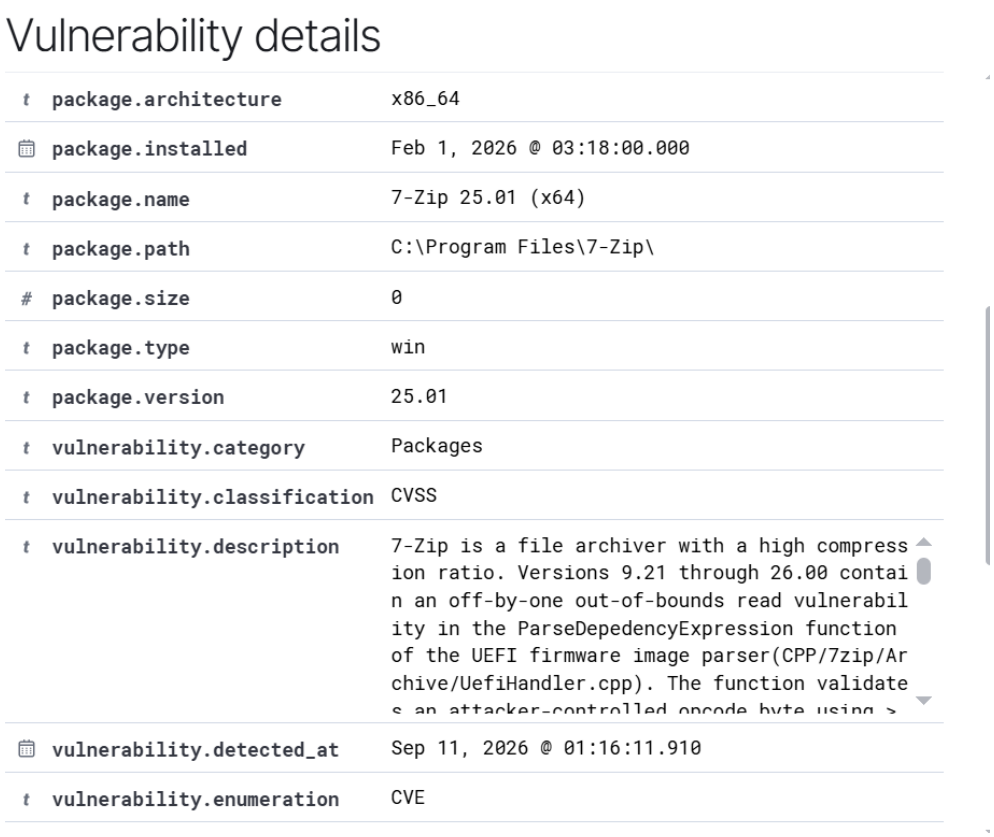


Figure 13.3: Details of the chosen finding (upper part): package 7-Zip 25.01 (x64), type `win`, path `C:\Program Files\7-Zip\`, package version 25.01, category Packages, classification CVSS, enumeration CVE, detected on 11 Sep 2026 at 01:16:11.910. The description reads "7-Zip is a file archiver with a high compression ratio. Versions 9.21 through 26.00 contain an off-by-one out-of-bounds read vulnerability in the ParseDependencyExpression function of the UEFI firmware image parser …" and is cut off in the screenshot.

Field	Value	Source
CVE ID	`CVE-2026-48111`	Author's notes and the search box in Figure 13.9
Severity	High	Author's notes; consistent with the dashboard's High tile
CVSS base score and version	7.1, CVSS 3.1	Author's notes (not visible in Figure 13.3)
Affected software	7-Zip 25.01 (x64)	Figure 13.3
Installed path	`C:\Program Files\7-Zip\`	Figure 13.3
Vulnerable condition	Package version below 26.01	Author's notes; the description in Figure 13.3 gives the range 9.21 to 26.00
Fixed version	26.01	Author's notes
First detected	11 Sep 2026, 01:16:11.910	Figure 13.3

Field	Value	Source
Data source	National Vulnerability Database	Author's notes

What the vulnerability is. According to the record, an off-by-one out-of-bounds read exists in the code that parses UEFI firmware images, and processing a specially crafted firmware image could cause a denial of service or limited information disclosure (author's notes).

Reading the detection date. The finding was first recorded on 11 September 2026, earlier than the documented phases, which began in October. So the finding already existed in Wazuh before it was investigated here. The timestamp shows when Wazuh matched the package, not when 7-Zip was installed or exploited.

Initial assessment. The installed version fell inside the vulnerable range. That is all the finding establishes. It does not show that an attacker used the flaw, and it does not show that the endpoint is compromised.

13.5 Step 3: verifying the installed version

Before changing anything, the version was checked with a source independent of Wazuh, the Windows uninstall registry:

```
Get-ItemProperty `
  'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\*',
  'HKLM:\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Uninstall\*' |
  Where-Object { $_.DisplayName -like '*7-Zip*' } |
  Select-Object DisplayName, DisplayVersion, InstallLocation
```

```
DisplayName       DisplayVersion InstallLocation
-----------       -------------- ---------------
7-Zip 25.01 (x64) 25.01          C:\Program Files\7-Zip\
```

Figure 13.4: Before the update: `7-Zip 25.01 (x64)`, version 25.01, `C:\Program Files\7-Zip\`. Windows and Wazuh agree on the vulnerable version.

Both uninstall hives are queried because a 64-bit program normally registers in the first one and a 32-bit program in the second; checking both avoids missing the package.

13.6 Step 4: remediation

7-Zip was updated from the official site (https://www.7-zip.org/) and the same query was repeated:

```
DisplayName       DisplayVersion InstallLocation
-----------       -------------- ---------------
7-Zip 26.04 (x64) 26.04          C:\Program Files\7-Zip\
```

Figure 13.5: After the update: `7-Zip 26.04 (x64)`, version 26.04, same install location. 26.04 is newer than the fixed version 26.01 named in the finding.

The local check proves that **Windows** now reports 26.04. It does not yet prove that **Wazuh** knows. The remediation was therefore not treated as verified until the Wazuh inventory and the vulnerability state had been checked.

13.7 Step 5: the finding stayed after the fix

After the update, `CVE-2026-48111` was still listed in Wazuh (recorded in the lab notes). This is a normal and instructive situation: the endpoint and the SIEM disagreed, and the analyst has to find out why instead of assuming either side is right.

5a. What does the Wazuh inventory say? Under IT Hygiene, Software Packages, Wazuh still showed the old package:

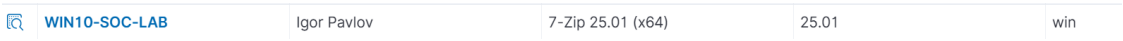


Figure 13.6: Wazuh package inventory still listing `7-Zip 25.01 (x64)`, version 25.01, for `WIN10-SOC-LAB`, publisher Igor Pavlov, type `win`, while the Windows registry already reported 26.04.

The scanner was working from this stale record, which explains why the finding was still listed. It does not yet explain why the record was stale.

5b. Is the inventory collection switched on? The agent's `ossec.conf` contains:

```
<wodle name="syscollector">
  <disabled>no</disabled>
  <interval>1h</interval>
  <scan_on_start>yes</scan_on_start>
</wodle>
```

Syscollector is enabled, runs hourly and also scans when the agent starts. So the configuration is not the problem. In theory a stale record should clear within about an hour once the agent can report.

5c. Did the module run? The agent log was searched:

```
Get-Content -Path 'C:\Program Files (x86)\ossec-agent\ossec.log' -Tail 500 |
  Select-String -Pattern 'syscollector|inventory'

2026/10/13 13:57:52 wazuh-modulesd:syscollector: INFO: Module started.
2026/10/13 13:57:52 wazuh-modulesd:syscollector: INFO: Starting evaluation.
2026/10/13 13:58:17 wazuh-modulesd:syscollector: INFO: Evaluation finished.
```

Syscollector started and finished an evaluation (25 seconds). That shows the endpoint-side collection ran. It does not show that the result reached the manager. A "Module started" line is written when the module starts, which normally happens when the agent service starts; the notes do not say whether the agent was restarted manually, so what triggered this run is **not established**.

5d. Was the agent connected? The same log also contained connection errors:

```
ERROR: Unable to connect to '[<manager-ip>]:1514/tcp'
WARNING: Unable to connect to any server.
ERROR: Lost connection with manager.
WARNING: Process locked due to agent is offline.
```

So at some point the agent could not reach the manager, and inventory results could not be delivered while it was offline. The manager side and the network path were then checked:

Check	Command	Result
Manager service	`sudo systemctl status wazuh-manager --no-pager -l`	Active and running
Port listening	`sudo ss -lntp | grep ':1514'`	Port 1514 listening
Path from Windows	`Test-NetConnection <manager-ip> -Port 1514`	`TcpTestSucceeded : True`
Agent state (server)	`sudo /var/ossec/bin/agent_control -lc`	Agent 001 `WIN10-SOC-LAB` Active

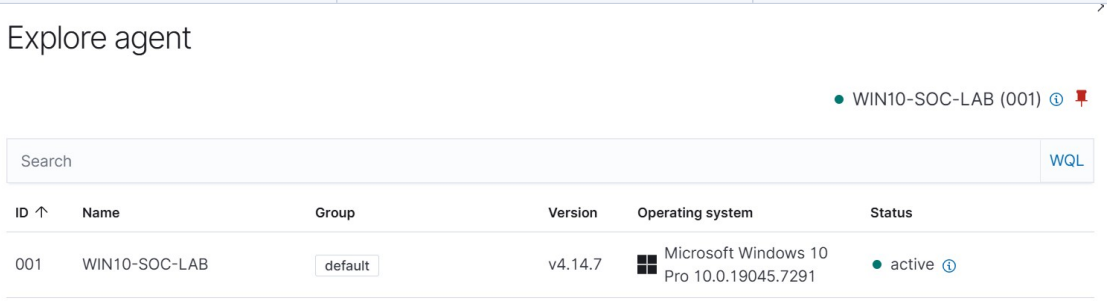


*Figure 13.7: Dashboard agent panel: `WIN10-SOC-LAB (001)`, group `default`, version v4.14.7, Microsoft Windows 10 Pro 10.0.19045.7291, status **active**. This is the dashboard view of the agent state; the `agent_control -lc` result listed above is recorded in the lab notes.*

What can be concluded. The agent had connection problems earlier; at the time of the checks the manager was running, the port was open, the path from Windows worked and the agent was active. The connection errors are a **plausible** reason for the stale record (the new inventory could not be delivered while the agent was offline), but they are **not proved** to be the only cause. Nothing in the evidence timestamps the errors against the update, so the cause of the stale record remains a hypothesis (open item 10).

13.8 Step 6: the inventory catches up and the finding closes

After the inventory refreshed, Software Packages showed the new version:

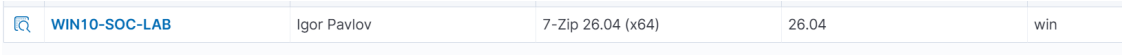


Figure 13.8: Wazuh package inventory now listing `7-Zip 26.04 (x64)`, version 26.04, the same value Windows reports (Figure 13.5).

Returning to Vulnerability Detection and searching for the CVE gave:

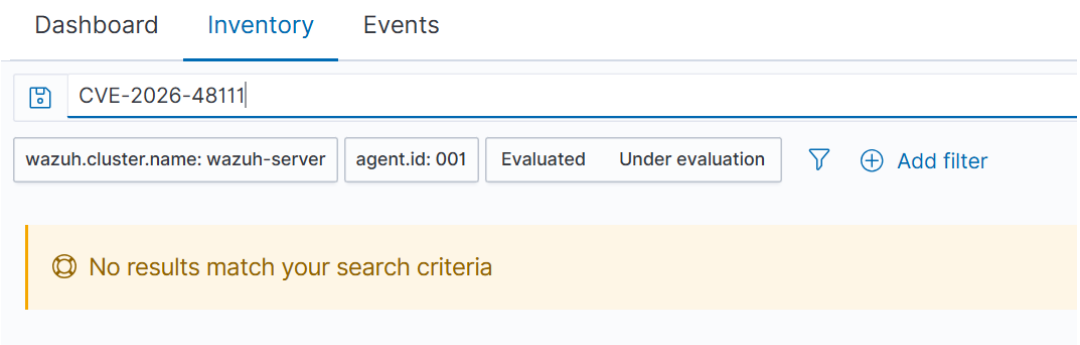


Figure 13.9: Vulnerability Detection, Inventory tab, search `CVE-2026-48111` with filters cluster `wazuh-server`, `agent.id: 001` and Evaluated / Under evaluation: "No results match your search criteria."

This is the verification the phase was aiming for: the software was updated, Wazuh's inventory reflects it, and the specific finding is gone. An empty result is meaningful here only because the earlier steps showed the finding *was* listed and because the search ran against the same agent and filters. Searching a mistyped CVE ID would also return nothing.

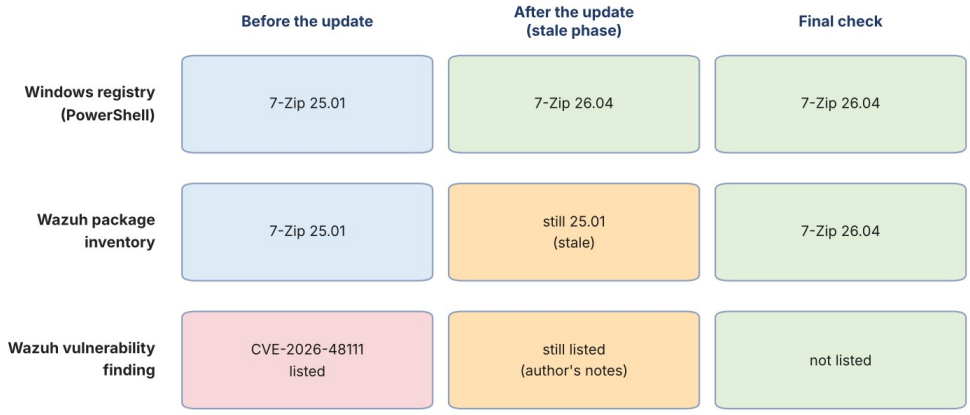


Figure 13.10: The same package seen from three places over time. The amber cells mark the stale phase: Windows already said 26.04 while Wazuh still reported 25.01. The "still listed" cell comes from the author's notes, not from a screenshot.

13.9 Analysis

Verdict: remediated and verified in the lab. The finding was a vulnerable-version match, not an incident.

Verification item	Result	Evidence
Original version identified	25.01	Figures 13.3, 13.4
Updated version verified locally	26.04	Figure 13.5
Syscollector enabled	Confirmed	`ossec.conf` (lab notes)
Syscollector evaluation completed	Confirmed	Agent log lines (notes)
Manager running, port 1514 listening	Confirmed	Command output (notes)
Windows to manager TCP connectivity	Successful	`Test-NetConnection` (notes)
Agent status	Active	Figure 13.7

Verification item	Result	Evidence
Wazuh inventory updated	Confirmed	Figure 13.8
`CVE-2026-48111` listed at the final check	**No**	Figure 13.9

What the evidence shows. The installed 7-Zip was inside the vulnerable range, it was updated to a version outside that range, and Wazuh's inventory and vulnerability view both reflect the update.

What it does not show.

- It does not show that every vulnerability on the endpoint is resolved. 90 other findings were not touched.
- It does not show that the host was clean. A vulnerability scanner reports weaknesses, not compromise.
- It does not prove why the inventory was stale (Section 13.7).
- It does not show how long the delay lasted; the intermediate states carry no times.

Exposure reasoning (judgment, not tested). Per the description, the flaw is triggered by processing a specially crafted UEFI firmware image. An attacker would have to get such a file to the user and have 7-Zip open it, and the stated impact is a denial of service or limited information disclosure. On a single-user lab machine that rarely handles firmware images, practical risk is therefore lower than the High label suggests, but the fix is cheap, so updating was the right call.

13.10 Proposed improvements (not deployed)

1. **Triage the remaining findings before fixing any.** Sort the 22 High findings by package, mark which software is actually used, and fix the exposed and widely used ones first (browsers, archivers, PDF readers, runtimes) before the rarely used ones.
2. **Keep a short remediation record per finding:** CVE, package, version before and after, date, who fixed it, and which dashboard view confirmed closure. Figure 13.10 is a template for that record.
3. **Always verify with two sources.** Check the host (registry or `Get-Package`) and the SIEM (inventory and finding). When they disagree, check the agent state and the manager connection before concluding anything.
4. **Do not wait for the hourly scan when a fix is urgent.** Restarting the agent service triggers a Syscollector scan because `scan_on_start` is `yes`. This is a configuration-based expectation that was not tested here.
5. **Watch agent connectivity as part of detection health.** An agent that is offline sends no inventory, no alerts and no FIM events, so a connection error in the agent log is a finding in its own right.

13.11 Phase 11 status and lessons

Item	Status
Findings overview captured	Complete (Figure 13.2)
One High finding investigated	Complete (Figure 13.3)

Item	Status
Installed version verified before and after	Complete (Figures 13.4, 13.5)
Stale inventory investigated	Complete; **cause not established**
Agent health checked	Complete
Remediation verified in Wazuh	Complete (Figures 13.8, 13.9)
Remaining 90 findings	Not triaged

Lessons

1. A vulnerability finding is a weakness, not proof of exploitation.
2. Verify the installed version with an independent source before and after the fix.
3. A finished installer is not a finished remediation: the scanner has to stop reporting the finding.
4. A stale SIEM record is not the same as an unfixed host. Check the inventory, then the agent, then the connection.
5. Connection errors in a log make a good hypothesis, not a proved cause, unless their timing matches.
6. An empty search result is only evidence when the same search showed the finding earlier.

Portfolio value: this phase shows a complete vulnerability-management cycle (find, confirm, fix, verify) and the troubleshooting needed when the SIEM lags behind the endpoint.

14. Custom Rules Reference

14.1 Live configuration (captured 11 October 2026)

This is the content of `/var/ossec/etc/rules/local_rules.xml` as shown in Figures 10.4 and 10.5, transcribed without changes. The sample rule 100001 that ships with Wazuh is omitted.

```
<group name="local,syslog,sshd,">

  <rule id="100101" level="10">
    <if_sid>61603</if_sid>
    <field name="win.eventdata.parentImage" type="pcre2">(?i)cmd.exe</field>
    <field name="win.eventdata.image" type="pcre2">(?i)powershell.exe</field>
    <description>PowerShell spawned by cmd.exe</description>
    <group>sysmon,powershell,process_creation,</group>
  </rule>

  <rule id="100102" level="10">
    <if_sid>61603</if_sid>
    <field name="win.eventdata.image" type="pcre2">(?i)\\schtasks\.exe$</field>
    <field name="win.eventdata.commandline" type="pcre2">(?i)\s/create(\s|$)</field>
    <description>Scheduled task created using schtasks.exe</description>
    <mitre>
      <id>T1053.005</id>
    </mitre>
    <group>sysmon,persistence,scheduled_task.</group>
  </rule>

  <rule id="100103" level="8">
    <if_group>sysmon_event3</if_group>
    <field name="win.eventdata.image" type="pcre2">(?i)\\powershell\.exe$</field>
    <field name="win.eventdata.protocol">^tcp$</field>
    <description>powershell initiated a TCP network connection to $
(win.eventdata.destinationIp):$(win.eventdata.destinationPort)</description>
    <group>sysmon_event3,powershell,network_connection,</group>
  </rule>

</group>
```

Rule	Level	Purpose	Status
100101	10	PowerShell spawned by `cmd.exe`	Confirmed working by the author
100102	10	Scheduled task created with `schtasks.exe /create` (T1053.005)	Confirmed working by the author
100103	8	PowerShell initiated a TCP connection	**Unresolved**: no matching alert verified. See Sections 8.7 and 10.12

14.2 Proposed hardened versions (not deployed, not tested)

These are the changes suggested by the Phase 8 review (Section 10.8). They must be validated with `wazuh-analysisd -t` and a controlled test before they replace the live rules.

```
<group name="lab_custom_detections,">
```

```
  <rule id="100101" level="10">
    <if_sid>61603</if_sid>
    <field name="win.eventdata.parentImage" type="pcre2">(?i)\\cmd\.exe$</field>
    <field name="win.eventdata.image" type="pcre2">(?i)\\powershell\.exe$</field>
    <description>PowerShell spawned by cmd.exe</description>
    <group>sysmon,powershell,process_creation,</group>
  </rule>

  <rule id="100102" level="10">
    <if_sid>61603</if_sid>
    <field name="win.eventdata.image" type="pcre2">(?i)\\schtasks\.exe$</field>
    <field name="win.eventdata.commandLine" type="pcre2">(?i)\s/create(\s|$)</field>
    <description>Scheduled task created using schtasks.exe</description>
    <mitre>
      <id>T1053.005</id>
    </mitre>
    <group>sysmon,persistence,scheduled_task,</group>
  </rule>

</group>
```

Changes: anchored and escaped patterns for 100101, the field spelled as the decoder writes it (`commandLine`) for 100102, the stray period in the group tag replaced by a comma, and a meaningful outer group name. Which spelling of the field name is required is still to be confirmed by the retest in Section 10.12.

15. Commands Reference

Purpose	Command
Hunting test	`powershell.exe -NoProfile -Command "Get-Date"`
Persistence test	`schtasks /Create /TN "Wazuh-Day03-Test" /TR "cmd.exe /c echo DAY03-PERSISTENCE-TEST" /SC ONLOGON /F`
Correlation test	`powershell.exe -NoProfile -Command "Resolve-DnsName example.com; curl.exe https://example.com"`
Validate Wazuh rules	`sudo /var/ossec/bin/wazuh-analysisd -t`
Look for an alert by rule ID	`grep '"id":"100103"' /var/ossec/logs/alerts/alerts.json`
DNS test (Phase 6)	`nslookup example.com`
HTTPS test (Phase 6)	`curl.exe https://example.com`
Find an archived Sysmon Event 3 (Phase 7)	`grep '"eventID":"3"' /var/ossec/logs/archives/archives.json | grep -m 1 '"destinationIp":"8.8.8.8"'`
Search alerts for rule 61605 (Phase 7)	`grep '"id":"61605"' /var/ossec/logs/alerts/alerts.json`
Review FIM settings (Phase 10)	`Select-String -Path "C:\Program Files (x86)\ossec-agent\ossec.conf" -Pattern "<syscheck>","<directories","<frequency>"`
Hash a file (Phase 10)	`Get-FileHash $path -Algorithm SHA256`
FIM test: modify and delete (Phase 10)	`Add-Content -Path $path -Value "..."`, `Remove-Item -LiteralPath $path`, then `Test-Path $path`
Installed version of a package (Phase 11)	`Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\*','HKLM:\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Uninstall\*' | Where-Object { $_.DisplayName -like '*7-Zip*' } | Select-Object DisplayName, DisplayVersion, InstallLocation`
Search the agent log for inventory messages (Phase 11)	`Get-Content 'C:\Program Files (x86)\ossec-agent\ossec.log' -Tail 500 | Select-String 'syscollector|inventory'`
Test the path to the manager (Phase 11)	`Test-NetConnection <manager-ip> -Port 1514`

Purpose	Command
Manager service and port (Phase 11)	`sudo systemctl status wazuh-manager --no-pager -l, sudo ss -lntp | grep ':1514'`
List connected agents (Phase 11)	`sudo /var/ossec/bin/agent_control -lc`

16. Key Lessons

- EncodedCommand is not malware, and PowerShell is not malware.
- A temp file is not malware, and file creation (Event 11) is not execution.
- A DNS query (Event 22) is not malicious DNS, and a network event (Event 3) is not proof of attack.
- A MITRE mapping is not a compromise, and a high rule level is not a confirmed malicious verdict.
- The reverse also holds: several suspicious indicators, plus supporting evidence, plus a clear timeline raise confidence.
- Detections must be built from the real telemetry, not assumptions about it.
- An event in the archive is not an alert, a valid configuration is not a working detection, and an unresolved problem must be recorded as unresolved, not as a fix.
- Document the live configuration, not the one you remember; where they differ, record the difference.
- A FIM alert proves a change, not intent and not which process made it; correlate it with process telemetry.
- Verify tool output with an independent check (for example a hash computed by PowerShell) and say what was not independently verified.
- A vulnerability finding is a weakness, not an incident; confirm the installed version with an independent source before and after the fix.
- A stale SIEM record is not the same as an unfixed host: check the inventory, then the agent, then the connection, and do not call a log error the cause without matching times.
- A silent rule is not always a broken rule: read the rule’s own conditions (level, process, protocol) before calling a detection failed.
- A failed logon is not an attack and a successful logon is not proof of legitimacy: read the sequence, the logon type and what happens after.

17. MITRE ATT&CK Coverage

Coverage is graded honestly: **Detection** means a custom rule was written and tested; **Telemetry** means the events were observed and investigated but no custom rule exists; **Reasoning** means it was only analysed in a scenario.

Tactic	Technique	Lab activity	Coverage
Execution	T1059.001 PowerShell	PowerShell hunts, parent-child alerts, encoded commands (Phases 2 to 4)	Detection (rule 100101, built-in 92027) and telemetry
Execution	T1059.003 Windows Command Shell	cmd.exe launching PowerShell, process tree (Phase 4)	Detection (rule 100101)
Persistence	T1053.005 Scheduled Task	`schtasks /Create` test (Phase 3); the persisting task reviewed in Phase 9	Detection (rule 100102, tagged in the rule); alert investigated, execution evidence gap
Persistence	T1547.001 Startup Folder	A harmless text file added to the monitored Startup folder (Phase 10); it does not execute	Detection by built-in FIM rule 554 (level 5); no custom rule
Defense Evasion	T1070.004 File Deletion	Deletion of the test file detected (Phase 10)	Telemetry (built-in FIM deletion alert, level 7)
Defense Evasion	T1027 Obfuscated Files or Information (encoded commands)	`-EncodedCommand` hunt, decoded to `Get-Date` (Phase 4)	Telemetry and reasoning, no dedicated rule
Command and Control	T1071.004 DNS	Event 22 queries correlated by ProcessGuid (Phases 4, 6); an Event 3 to 8.8.8.8:53 reviewed and deliberately not mapped (Phase 7)	Telemetry
Command and Control	T1071.001 Web Protocols	`curl.exe` HTTPS request (Phase 6)	Telemetry, rule 100103 unresolved
Credential Access	T1110 Brute Force	Failed-then-successful logon timeline (Phase 5)	Telemetry and reasoning, no rule; the test itself was benign
Initial Access / Persistence	T1078 Valid Accounts	Reading a 4624 success in context (Phase 5)	Reasoning
Discovery	T1046 Network Service Discovery	Internal scanning scenario (Phase 6, hypothetical)	Reasoning only

A MITRE tag describes a technique. It does not prove that the technique was used maliciously.

Phase 11 adds no technique row on purpose: a CVE finding is an exposure, not observed attacker behaviour, and no exploitation was seen.

18. Detection Catalogue

Rule	Logic	Test case	Status	Likely false positives	Tuning idea
100101 (level 10)	Sysmon Event 1, parent `cmd.exe`, image `powershell.exe`	`cmd` then `powershell` (Phase 4 process tree)	Confirmed working by the author; live patterns are looser than intended (Section 10.8)	Admin scripts, installers and logon scripts that call PowerShell from a batch file	Exclude known script paths or signed parents; raise only with an encoded command or a network event
100102 (level 10, T1053.005)	Event 1, image `schtasks.exe`, command line contains `/create`	`schtasks /Create ... /SC ONLOGON` (Phase 3)	Confirmed working: alert shown in Figures 5.7 and 11.2	Software updaters and IT management tools creating tasks	Allow-list known task names and parents; raise when the task action runs PowerShell or an encoded command
100103 (level 8)	Event 3 group `sysmon_event3`, image `powershell.exe`, protocol `tcp`	`curl.exe` and `nslookup` tests (Phase 6)	**Unresolved**: no alert on a real Event 3	Almost any PowerShell web request, so it would be noisy	Fix the match first, then restrict to non-private destinations or unusual ports
92027 (level 4, built-in)	PowerShell spawned PowerShell	`powershell -NoProfile -Command "Get-Date"`	Fired (Phase 3)	Very common	Informational only
60122 / 60118 (built-in)	Failed logon / workstation logon success	Controlled wrong-password test (Phase 5)	Fired (Phase 5)	Mistyped passwords, stale service credentials	Use frequency rules (several failures from one source) rather than single events
554 (level 5, built-in)	FIM: file added to a monitored path	Test file added to the Startup folder (Phase 10)	Fired (Figures 12.5 to 12.8)	Installers and updaters adding files	Raise severity for executables and scripts in Startup (Section 12.10, sketch)
FIM modified (level 7, built-in; ID to confirm)	Size, time or checksum changed on a monitored file	Appended one line to the test file	Fired (Figures 12.11, 12.12)	Software updates, configuration edits	Alert on sensitive paths first; find the responsible process with who-data or Sysmon
FIM deleted (level 7, built-in; ID to confirm)	Monitored file removed	`Remove-Item` on the test file	Fired (Figures 12.14, 12.15)	Cleanup and uninstalls	Review deletions of system binaries and logs first

Rule	Logic	Test case	Status	Likely false positives	Tuning idea
Vulnerability Detection (built-in module, not a rule)	Installed package version falls in a CVE's vulnerable range (here below 26.01)	7-Zip 25.01 installed (Phase 11)	Finding listed; cleared after the update to 26.04 and an inventory refresh (Figures 13.8, 13.9)	Fixes backported without a version change; stale inventory	Triage by CVSS, exposure and real use of the software; confirm closure in both the host and the SIEM

19. Sample Incident Report (Lab Exercise)

This report applies the investigation workflow to the real Phase 5 evidence. A second, longer case (the scheduled task from Phase 3) is written up in Section 11. It is written as an analyst would write it, and the incident is a controlled test, not a real intrusion.

Field	Value
Report type	Authentication alert triage (lab exercise)
Host / agent	`WIN10-SOC-LAB` (Agent 001)
Account	`lab-user` (local)
Detection	Wazuh rules 60122 (three times) then 60118
Final verdict	**Benign, controlled authentication testing**

Summary. Three failed logons were followed by one successful logon for the same account within about 19 seconds.

Timeline (times as displayed in the dashboard).

Time	Event
15:16:35.903	4625 failed logon, bad password
15:16:40.118	4625 failed logon, bad password
15:16:45.748	4625 failed logon, bad password
15:16:54.506	4624 successful logon, logon type 2

Evidence. Same account, logon type 2 (interactive), source 127.0.0.1, same caller process (`svchost.exe`, PID 0x984), failure status 0xC000006D / 0xC000006A, elevated token on success.

Analysis. Failures about 5 seconds apart from the local console, no remote source, one account, then success. Remote logon types (3 or 10), many accounts or an external source address would have raised the priority.

Not checked / unknown. Post-login process activity was not reviewed for this window, because the test was known to be controlled.

Recommendation. No action. In a real environment, review process creation, scheduled tasks and network connections after the 4624 before closing, and confirm the user recognises the activity.

20. Evidence Index

Every screenshot used in this report is listed with its file name, so each piece of evidence can be traced to the phase that uses it.

Phase	File	Shows
1	`01-agent-active.png`	Agent active in the dashboard (Figure 3.1)
3	`01-custom-rule-100101.png`	Rule 100101
3	`02b-powershell-process-evidence.png`	Process evidence
3	`03-scheduled-task-created.png`	Task created
3	`04-scheduled-task-details.png`	Task details (schtasks /Query)
3	`06-scheduled-task-sysmon-event1.png`	schtasks Event 1
3	`07-custom-rule-scheduled-task.png`	Rule 100102 (intermediate edit)
3	`08-scheduled-task-custom-alert.png`	Alert details, fresh test
3	`correlation/event1-process.png`	Correlation Event 1
3	`correlation/event22-dns.png`	Correlation Event 22
4	`01-process-investigation.png`	cmd.exe Event 1 details
4	`02-process-tree.png`	Three-process tree
4	`03-powershell-encoded-hunt.png`	Encoded PowerShell Event 1
4	`04-correlation-event1.png`	Correlation: Event 1 process
4	`05-correlation-event11.png`	Correlation: Event 11 temp file
4	`06-correlation-event22.png`	Correlation: Event 22 DNS query
5	`01-failed-logon-4625.png`	Event 4625 failed logon
5	`02-successful-logon-4624.png`	Event 4624 successful logon
5	`03-authentication-correlation.png`	4625 x3 then success timeline
6	`01-curl-event1.png`	curl.exe Event 1

Phase	File	Shows
6	`02-custom-rule-100103.png`	Rule 100103 in local_rules.xml
6	`03-powershell-event22.png`	PowerShell Event 22 (8 Oct)
6	`04a-nslookup-event1-top.png`	nslookup Event 1, upper part
6	`04b-nslookup-event1-bottom.png`	nslookup Event 1, lower part
6	`05a-nslookup-event3-top.png`	nslookup Event 3, upper part
6	`05b-nslookup-event3-bottom.png`	nslookup Event 3, lower part
6	`06b-archived-event3-opera.png`	Archived Event 3, Opera GX UDP to 8.8.8.8 (10 Oct)
7	`01-sysmon-event3-archive.png`	Archived Event 3 (same image as phase-6/06b)
8	`01-event1-cmd-powershell.png`	Event Viewer: cmd.exe starts powershell.exe (11 Oct)
8	`02-archived-event1.png`	Archived Event 1, camelCase field names
8	`03-local-rules-live.png`	Live local_rules.xml
8	`04-local-rules-structure.png`	Rule file structure by line number
9	`01-rule-100102-alert.png`	Wazuh row for rule 100102 (7 Oct)
9	`02-sysmon-schtasks-create.png`	Sysmon Event 1, schtasks /Create, Test-4
9	`03-scheduled-task-query.png`	`schtasks /Query /V /FO LIST` output
10	`01-fim-config-syscheck.png`	syscheck block: FIM enabled, 12 h frequency
10	`02-fim-config-directories.png`	Monitored paths, Startup folder real time
10	`03-create-test-file.png`	Create test file, hash command
10	`04-fim-file-added-row.png`	Wazuh row: file added, level 5
10	`05-fim-file-added-details.png`	Added alert details, upper part
10	`06-fim-file-added-rule-fields.png`	Added alert: rule 554 and hashes

Phase	File	Shows
10	07-fim-file-added-syscheck.png	Added alert: size, owner, permissions
10	08-fim-hash-verification.png	PowerShell SHA-256 matches Wazuh
10	09-modify-test-file.png	Append line, size 124
10	10-fim-file-modified-row.png	Wazuh row: file modified, level 7
10	11-fim-file-modified-fulllog.png	Modified alert: before and after values
10	12-delete-test-file.png	Remove-Item, Test-Path False
10	13-fim-file-deleted-row.png	Wazuh row: file deleted, level 7
10	14-fim-file-deleted-details.png	Deleted alert: last known state
11	01-vulnerability-overview.png	Dashboard: 0 / 22 / 57 / 12 findings
11	02-high-vulnerability-7zip.png	Finding details: 7-Zip 25.01
11	03-installed-version-verification.png	Registry query before update: 25.01
11	04-7zip-remediation-verified.png	Registry query after update: 26.04
11	06-agent-active-status.png	Agent panel: active, v4.14.7
11	07-package-inventory-stale-record.png	Wazuh inventory still showing 25.01
11	08-updated-package-inventory.png	Wazuh inventory showing 26.04
11	09-cve-remediation-verification.png	CVE search: no results